

The Socket Exchange

2024CS10411 Mridul Chhipa | 2024CS10499 Harsh Verma

Experiment Report
COL 334 – Computer Networks, Assignment 2

Contents

1	Experimental Setup	1
2	Implementation Decisions	2
3	Experiment 1 – Listening and Connected Sockets	2
4	Experiment 2 – Observing TCP Connection States	3
5	Experiment 3 – TCP as a Byte Stream	5
6	Experiment 4 – One Client Should Not Stall the Others	7
7	Experiment 5 – Multiple Clients and I/O Multiplexing	8
8	Experiment 6 – FIN vs. RST: Orderly and Abrupt Termination	12
9	Experiment 7 – Backpressure and the Slow Receiver	14
10	Experiment 8 – Unexpected Client Disconnection	16
11	Summary of Tools Used	19
A	Bonus – Connection Scalability and I/O Design	19
A.1	Setup and tooling	19
A.2	Changes made to the Exchange Server	20
A.3	Client-side source-IP rotation (for $N > 55\,000$)	20
A.4	Full measurement table	21
A.5	Screenshots	21
A.6	Answers to Section 6.9 questions	27
A.7	Notes and caveats	28

1 Experimental Setup

All processes – the Exchange Server, the harness-generated clients and the observation tools – run inside a single FreeBSD virtual machine. The server binds `127.0.0.1:5000`, so every connection in this report is a loopback connection and all packet captures are taken on `lo0`.

Launchers. The harness always starts the server through the standard launcher, so it never needs to know the implementation language:

```
$ cat server/run-server
#!/bin/sh
exec "$(dirname "$0")/../exchange_server" "$@"

$ cat client/run-trader
#!/bin/sh
exec "$(dirname "$0")/../trader_client" "$@"
```

```
$ cat client/run-market-data
#!/bin/sh
exec "$(dirname "$0")/../../market_data_client" "$@"
```

Running an experiment. Each experiment is started from the submission root with

```
$ python3 experiment.py <experiment-number>
```

and observed from one or more additional SSH sessions into the same VM while the harness is in its observation phase.

2 Implementation Decisions

- **Concurrency / I/O model.** Single process, single thread, one event loop on `kqueue/kevent`; the only blocking syscall is `kevent()`. No thread or process per connection.
Why: `kqueue` is $O(1)$ per ready event (versus $O(N)$ for `select/poll` and `FD_SETSIZE=1024` for `select`), avoids the $\sim 1\text{--}2\text{ MB}$ per-thread stack that caps thread-per-connection at a few thousand clients, and removes the shared-state locking that a multi-threaded design would need for the order book – letting the server scale to 70k idle connections at 0% CPU (Appendix A).
- **Non-blocking sockets.** Every socket – listening and accepted – is set `O_NONBLOCK`, so `accept()`, `read()` and `write()` never stall.
Why: a single blocked syscall would freeze every other client; non-blocking I/O keeps the event loop responsive.
- **Edge-triggered filters.** Each accepted socket is registered with `EVFILT_READ` and `EVFILT_WRITE` + `EV_CLEAR`; `accept()` and `read()` loop until `EAGAIN`.
Why: edge-triggered notifications fire once per readiness transition, so the kernel does not re-report the same fd on every `kevent()` return, cutting wake-ups under load.
- **Message framing.** Line-oriented protocol over a TCP byte stream: a per-connection input buffer is scanned for `'\n'` after each `read()`; partial lines are compacted and retained.
- **Output buffering / back-pressure.** Short or `EAGAIN` writes spill into a per-connection output buffer drained on `EVFILT_WRITE`; a slow reader only bloats its own buffer.
- **Role assignment.** A connection has no role until the first command: `LOGIN` promotes it to `Trader`, `SUBSCRIBE` to `Market-Data`; cross-role commands are rejected with `ERROR`.
- **Disconnect handling.** `EVFILT_READ+EV_EOF` with `read()==0` causes the server to remove both filters, release the slot, and `close()`; no other connection is touched.
- **Graceful shutdown.** `SIGINT/SIGTERM` interrupts `kevent()` with `EINTR`; the loop closes all connections, the `kqueue` and the listening socket before exiting.

3 Experiment 1 – Listening and Connected Sockets

Approach and tools

The harness (`python3 experiment.py 1`) leaves one idle client connected. Sockets are enumerated with `sockstat` (owner PID / FD) and `netstat` (TCP-state view):

```
$ sockstat -4 -l | grep -E 'USER|exchange'
$ sockstat -4 -c | grep -E 'USER|exchange|python'
$ netstat -an -p tcp | grep 5000
```

Observations

```
zfs list -o name mypool/usr

More columns and their order can be defined by separating them with commas:

zfs list -o mountpoint,name,avail

-- Benedict Reuschling <bcr@FreeBSD.org>
[harsh@ragnarok ~]$ cd compiled
[harsh@ragnarok ~/compiled]$ sockstat -4 -l
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS    FOREIGN ADDRESS
harsh     exchange_s 1874 3   tcp4   127.0.0.1:5000    *:*
harsh     node      1002 14  tcp4   127.0.0.1:36880   *:*
harsh     node      964 14  tcp4   127.0.0.1:45866   *:*
root      sshd      813 7    tcp4   *:22              *:*
root      syslogd   715 7    udp4   *:514              *:*
[harsh@ragnarok ~/compiled]$ sockstat -4 -c
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS    FOREIGN ADDRESS
harsh     exchange_s 1874 5   tcp4   127.0.0.1:5000    127.0.0.1:47532
harsh     python3.12 1873 3   tcp4   127.0.0.1:47532   127.0.0.1:5000
harsh     sshd-sessi 1387 7   tcp4   10.0.2.15:22      10.0.2.2:54901
root      sshd-sessi 1382 7   tcp4   10.0.2.15:22      10.0.2.2:54901
harsh     sshd-sessi 1290 7   tcp4   10.0.2.15:22      10.0.2.2:58990
root      sshd-sessi 1283 7   tcp4   10.0.2.15:22      10.0.2.2:58990
harsh     node      1186 15  tcp4   127.0.0.1:36880   127.0.0.1:30236
harsh     node      1186 19  tcp4   10.0.2.15:29100   54.82.101.0:443
harsh     sshd-sessi 1044 7   tcp4   10.0.2.15:22      10.0.2.2:53578
harsh     sshd-sessi 1044 8   tcp4   127.0.0.1:30236   127.0.0.1:36880
harsh     sshd-sessi 1044 10  tcp4   127.0.0.1:38851   127.0.0.1:36880
root      sshd-sessi 1038 7   tcp4   10.0.2.15:22      10.0.2.2:53578
harsh     node      1002 15  tcp4   127.0.0.1:36880   127.0.0.1:38851
[harsh@ragnarok ~/compiled]$
```

Figure 1: `sockstat -4 -l` and `sockstat -4 -c`: the full listening and connected IPv4 socket tables while the experiment client is connected. The Exchange Server is PID 1874.

```
[harsh@ragnarok ~/compiled]$ sockstat -4 -l | grep -E 'USER|exchange'
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS    FOREIGN ADDRESS
harsh     exchange_s 1874 3   tcp4   127.0.0.1:5000    *:*
[harsh@ragnarok ~/compiled]$ sockstat -4 -c | grep -E 'USER|exchange|python'
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS    FOREIGN ADDRESS
harsh     exchange_s 1874 5   tcp4   127.0.0.1:5000    127.0.0.1:47532
harsh     python3.12 1873 3   tcp4   127.0.0.1:47532   127.0.0.1:5000
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      0      0 127.0.0.1.5000      127.0.0.1.47532     ESTABLISHED
tcp4      0      0 127.0.0.1.47532     127.0.0.1.5000     ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
[harsh@ragnarok ~/compiled]$
```

Figure 2: The same information filtered to the server and its client, followed by `netstat -an -p tcp | grep 5000`: two ESTABLISHED rows (one per endpoint of the loopback connection) and one LISTEN row.

The server (PID 1874) owns two TCP sockets: **fd 3** (127.0.0.1:5000, peer `:*`, LISTEN) and **fd 5** (127.0.0.1:5000, peer 127.0.0.1:47532, ESTABLISHED). The client process holds the mirror of the second socket, which is why `netstat` lists the connection twice on loopback.

Answer. The server owns one listening socket (fd 3, half-association 127.0.0.1:5000 → `:*`, state LISTEN) and one connected socket (fd 5, full four-tuple 127.0.0.1:5000 ↔ 127.0.0.1:47532, state ESTABLISHED). The listening socket carries no data; `accept()` creates a new descriptor for each client while the listener keeps serving the port.

4 Experiment 2 – Observing TCP Connection States

Approach and tools

The harness connects a client, holds it idle for 10 s (phase 1), closes it, and keeps the server alive for another 15 s (phase 2). Two observers run in parallel: `netstat` snapshots taken at each announced phase, and `tcpdump` on `lo0`.

```
$ netstat -an -p tcp | grep 5000      # re-run at each phase transition
$ tcpdump -i lo0 -n -S tcp port 5000 # packet-level state transitions
```

Observations

```
[harsh@ragnarok ~/compiled]$ python3 experiment.py 2
=== Experiment 2: Observing TCP Connection States ===

Started Exchange Server (PID 3225).
Listening on 127.0.0.1:5000
Exchange Server is listening.

Experiment client: connected from 127.0.0.1:25275

Phase 1: the client is connected and idle.
Investigate the connection now.

Phase 2: closing the client connection.
Investigate the connection again.

The Exchange Server will remain running for 15 seconds.
Client 127.0.0.1:25275 disconnected

Experiment finished.
Stopping Exchange Server...
Shutting down, closing 10 connection slots
Exchange server closed successfully
[harsh@ragnarok ~/compiled]$
```

Figure 3: Harness output. Server PID 3225; the experiment client connects from 127.0.0.1:25275, stays idle for phase 1, and is closed at the start of phase 2.

```
root@ragnarok:/home/harsh/compiled # tcpdump -i lo0 -n -S tcp port 5000
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
19:04:49.730645 IP 127.0.0.1.34294 > 127.0.0.1.5000: Flags [S], seq 3435008793, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 1094157856 ecr 0], length 0
19:04:49.731268 IP 127.0.0.1.5000 > 127.0.0.1.34294: Flags [R.], seq 0, ack 3435008794, win 0, length 0
19:04:49.839859 IP 127.0.0.1.42356 > 127.0.0.1.5000: Flags [S], seq 3142239628, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 701203190 ecr 0], length 0
19:04:49.840010 IP 127.0.0.1.5000 > 127.0.0.1.42356: Flags [S.], seq 3410429500, ack 3142239629, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 4231954951 ecr 701203190], length 0
19:04:49.840178 IP 127.0.0.1.42356 > 127.0.0.1.5000: Flags [.], ack 3410429501, win 1277, options [nop,nop,TS val 701203190 ecr 4231954951], length 0
19:04:49.840322 IP 127.0.0.1.42356 > 127.0.0.1.5000: Flags [R.], seq 3142239629, ack 3410429501, win 0, options [nop,nop,TS val 701203190 ecr 4231954951], length 0
19:04:49.840627 IP 127.0.0.1.25275 > 127.0.0.1.5000: Flags [S], seq 407608313, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 2383817634 ecr 0], length 0
19:04:49.840656 IP 127.0.0.1.5000 > 127.0.0.1.25275: Flags [S.], seq 1534167980, ack 407608314, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 2305774737 ecr 2383817634], length 0
19:04:49.840681 IP 127.0.0.1.25275 > 127.0.0.1.5000: Flags [.], ack 1534167981, win 1277, options [nop,nop,TS val 2383817634 ecr 2305774737], length 0
19:04:59.857904 IP 127.0.0.1.25275 > 127.0.0.1.5000: Flags [F.], seq 407608314, ack 1534167981, win 1277, options [nop,nop,TS val 2383827652 ecr 2305774737], length 0
19:04:59.857971 IP 127.0.0.1.5000 > 127.0.0.1.25275: Flags [.], ack 407608315, win 1277, options [nop,nop,TS val 2305784755 ecr 2383827652], length 0
19:04:59.858238 IP 127.0.0.1.5000 > 127.0.0.1.25275: Flags [F.], seq 1534167981, ack 407608315, win 1277, options [nop,nop,TS val 2305784755 ecr 2383827652], length 0
19:04:59.858274 IP 127.0.0.1.25275 > 127.0.0.1.5000: Flags [.], ack 1534167982, win 1277, options [nop,nop,TS val 2383827652 ecr 2305784755], length 0
^C
13 packets captured
146 packets received by filter
0 packets dropped by kernel
root@ragnarok:/home/harsh/compiled #
```

Figure 4: tcpdump -i lo0 -n -S tcp port 5000: the three-way handshake at 19:04:49, ten seconds of complete silence, then the four-way teardown at 19:04:59.


```

[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      0      0 127.0.0.1.5000      127.0.0.1.25275      ESTABLISHED
tcp4      0      0 127.0.0.1.25275     127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      0      0 127.0.0.1.5000      127.0.0.1.25275      ESTABLISHED
tcp4      0      0 127.0.0.1.25275     127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
[harsh@ragnarok ~/compiled]$

```

Figure 5: `netstat -an -p tcp | grep 5000`, run at each stage – eight invocations in all. The first produces no output because it was run before the harness had started the server; the next two are taken during phase 1 and show both endpoints `ESTABLISHED` alongside the `LISTEN` socket; the following four are taken after phase 2 closed the connection and show only `LISTEN`; the last one, after the server exited, is empty again.

Mapping packets to states (Fig. 4, client port 25275):

Event on the wire	Client state	Server state
before anything	CLOSED	LISTEN (fd 3)
SYN sent	SYN_SENT	SYN_RCVD
SYN,ACK + final ACK	ESTABLISHED	ESTABLISHED (new fd)
idle phase (no packets)	ESTABLISHED	ESTABLISHED
client <code>close()</code> sends FIN	FIN_WAIT_1	CLOSE_WAIT
server ACKs the FIN	FIN_WAIT_2	CLOSE_WAIT
server sees EOF, <code>close()</code> sends FIN	TIME_WAIT [†]	LAST_ACK
client ACKs	(entry removed)	CLOSED

Two points about the `netstat` output:

- **The idle phase produces no packets.** An `ESTABLISHED` connection with no I/O is pure kernel state; keep-alives are off by default, so an idle client is invisible on the wire yet still `ESTABLISHED`.
- **No `TIME_WAIT` appears after the close.** This is a loopback-specific FreeBSD behaviour: `net.inet.tcp.nolocaltimewait` defaults to 1 on FreeBSD 14 and makes an endpoint with a local peer skip `TIME_WAIT`.

Answer. Every transition is caused by a packet or a syscall. The three-way handshake takes both ends `CLOSED/LISTEN` → `ESTABLISHED`; both stay `ESTABLISHED` through the 10s idle phase; the client's `close()` emits `FIN` (→ `FIN_WAIT_1/CLOSE_WAIT`), and the server's mirroring `FIN+ACK` closes both ends. Only the `LISTEN` socket survives.

5 Experiment 3 – TCP as a Byte Stream

Approach and tools

The harness sends the single protocol message `"LOGIN experiment_trader\n"` as four separate `send()` calls – `"LOGIN "`, `"experiment"`, `"_trader"`, `"\n"` – with a 200 ms pause between them.

The traffic is captured with payloads printed in ASCII so that the split is visible byte for byte:

```
$ tcpdump -i lo0 -n -A tcp port 5000
```

Observations

```
root@ragnarok:/home/harsh/compiled # tcpdump -i lo0 -n -A tcp port 5000
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
03:45:40.919723 IP 127.0.0.1.63659 > 127.0.0.1.5000: Flags [S], seq 1165544018, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 4079201088 ecr 0
], length 0
E..4..@.....Ex.R.....0...?.....
.#.....
03:45:40.919922 IP 127.0.0.1.5000 > 127.0.0.1.63659: Flags [R.], seq 0, ack 1165544019, win 0, length 0
E..4..@.....Ex.SP.....
03:45:41.028499 IP 127.0.0.1.57740 > 127.0.0.1.5000: Flags [S], seq 2004805460, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 2223285339 ecr 0
], length 0
E..4..@.....W..T.....0...?.....
...[....
03:45:41.028553 IP 127.0.0.1.57740 > 127.0.0.1.57740: Flags [S.], seq 402732527, ack 2004805461, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val
3056887093 ecr 2223285339], length 0
E..4..@.....5.W..U.....0...?.....
.4e5...[
03:45:41.028604 IP 127.0.0.1.57740 > 127.0.0.1.5000: Flags [.], ack 1, win 1277, options [nop,nop,TS val 2223285339 ecr 3056887093], length 0
E..4..@.....W..U..5.....(.....
...[.4e5
03:45:41.028773 IP 127.0.0.1.57740 > 127.0.0.1.5000: Flags [R.], seq 1, ack 1, win 0, options [nop,nop,TS val 2223285339 ecr 3056887093], length 0
E..4..@.....W..U..5.....(.....
...[.4e5
03:45:41.029183 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [S], seq 2738259465, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 571047032 ecr 0
], length 0
E..4..@.....?@...6. ....0...?.....
"
|x....
03:45:41.029323 IP 127.0.0.1.5000 > 127.0.0.1.16192: Flags [S.], seq 1564807594, ack 2738259466, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val
1872050621 ecr 571047032], length 0
E..4..@.....?@]E...6.
....0...?.....
o.9" |x
03:45:41.029370 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [.], ack 1, win 1277, options [nop,nop,TS val 571047032 ecr 1872050621], length 0
E..4..@.....?@...6.
]E.....(.....
"
|x.o.9.
03:45:41.029576 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [P.], seq 1:7, ack 1, win 1277, options [nop,nop,TS val 571047032 ecr 1872050621], length 6
E..4..@.....?@...6.
```

Figure 6: Start of the capture: the experiment connection from 127.0.0.1:16192 completes its handshake.

```
E..4..@.....?@...6.
]E.....(.....
"
|x.o.9.
03:45:41.029576 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [P.], seq 1:7, ack 1, win 1277, options [nop,nop,TS val 571047032 ecr 1872050621], length 6
E..4..@.....?@...6.
]E.....(.....
"
|x.o.9.LOGIN
03:45:41.070223 IP 127.0.0.1.5000 > 127.0.0.1.16192: Flags [.], ack 7, win 1277, options [nop,nop,TS val 1872050651 ecr 571047032], length 0
E..4..@.....?@]E...6.....(.....
o.9" |x
03:45:41.243816 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [P.], seq 7:17, ack 1, win 1277, options [nop,nop,TS val 571047248 ecr 1872050651], length 10
E..4..@.....?@...6..]E.....2.....
"
}Po.9.experiment
03:45:41.278234 IP 127.0.0.1.5000 > 127.0.0.1.16192: Flags [.], ack 17, win 1277, options [nop,nop,TS val 1872050871 ecr 571047248], length 0
E..4..@.....?@]E...6.....(.....
o.i." }P
03:45:41.456138 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [P.], seq 17:24, ack 1, win 1277, options [nop,nop,TS val 571047453 ecr 1872050871], length 7
E..4..@.....?@...6..]E...../.....
"
~o.i..trader
03:45:41.490000 IP 127.0.0.1.5000 > 127.0.0.1.16192: Flags [.], ack 24, win 1277, options [nop,nop,TS val 1872051071 ecr 571047453], length 0
E..4..@.....?@]E...6.i.....(.....
o.j." ~
03:45:41.671446 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [P.], seq 24:25, ack 1, win 1277, options [nop,nop,TS val 571047673 ecr 1872051071], length 1
E..5..@.....?@...6..]E.....).....
"
~o.j.
03:45:41.672246 IP 127.0.0.1.5000 > 127.0.0.1.16192: Flags [P.], seq 1:4, ack 25, win 1277, options [nop,nop,TS val 1872051262 ecr 571047673], length 3
E..7..@.....?@]E...6.."....4.....
o.<~" ~OK
03:45:41.710056 IP 127.0.0.1.16192 > 127.0.0.1.5000: Flags [.], ack 4, win 1277, options [nop,nop,TS val 571047714 ecr 1872051262], length 0
E..4..@.....?@...6..]E.....(.....
"
"o.<~
^C
18 packets captured
661 packets received by filter
0 packets dropped by kernel
root@ragnarok:/home/harsh/compiled #
```

Figure 7: The four fragments of one application message, each in its own TCP segment. The first three are acknowledged by a pure ACK; the fourth – the one carrying the newline – is acknowledged on the server's OK reply itself, which is the first byte the server sends in the whole exchange.

From Fig. 7 :

1. **The message crosses in four pieces.** Four segments of 6, 10, 7 and 1 bytes carry one 24-byte application message; sequence numbers are contiguous . Each arrives independently, so the server's `read()` returns four times.

2. **The server answers exactly once.** OK is emitted after the fourth segment. The first three segments draw a bare ACK; the fourth is acknowledged on the OK reply itself (ack 25 riding on the 3-byte payload). The server buffered the first three fragments and dispatched only when the '\n' arrived.

Answer. The server receives the message in four pieces (6, 10, 7, 1 bytes) and produces exactly one OK. TCP is a byte stream with no message boundaries: a sender's `send()` calls have no relationship to the receiver's `recv()` calls. Framing must be reconstructed by the receiver – here by buffering per connection and splitting at the '\n' delimiter.

6 Experiment 4 – One Client Should Not Stall the Others

Approach and tools

Client 1 sends a deliberately incomplete message ("LOGIN blocked_client", no trailing newline) and stays silent; two seconds later Client 2 connects and sends a complete message. The harness times Client 2's reply.

```
$ sockstat -4 -c | grep -E 'USER|exchange'
$ netstat -an -p tcp | grep 5000
$ procstat -k $(pgrep -f exchange_server)          # kernel stack
$ ps -H -o pid,tid,%cpu,wchan,comm -p $(pgrep -f exchange_server)
```

Observations

```
[harsh@ragnarok ~/compiled]$ python3 experiment.py 4
=== Experiment 4: One Client Should Not Stall the Others ===

Started Exchange Server (PID 1925).
Listening on 127.0.0.1:5000
Exchange Server is listening.

Client 1: connected from 127.0.0.1:61838

Client 1 will send an incomplete application message.
Client 1 will now remain silent.

Client 2: connected from 127.0.0.1:58885
Client 2 will send a complete application message.
6: LOGIN active_client
User active_client logged in from 127.0.0.1:58885

Client 2 response: 'OK'
Elapsed time: 0.005 seconds

Investigate the behavior of both TCP connections.
Press Ctrl-C when you are finished.
█
```

Figure 8: Harness output. Client 1 (port 61838) sends a partial message and falls silent; Client 2 (port 58885) then logs in and is answered 'OK' in **0.005 seconds**.

```
[harsh@ragnarok ~/compiled]$ sockstat -4 -c | grep -E 'USER|exchange'
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS    FOREIGN ADDRESS
harsh     exchange_s 1925 5    tcp4    127.0.0.1:5000    127.0.0.1:61838
harsh     exchange_s 1925 6    tcp4    127.0.0.1:5000    127.0.0.1:58885
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      0      0  127.0.0.1.5000      127.0.0.1.58885      ESTABLISHED
tcp4      0      0  127.0.0.1.58885     127.0.0.1.5000      ESTABLISHED
tcp4      0      0  127.0.0.1.5000      127.0.0.1.61838     ESTABLISHED
tcp4      0      0  127.0.0.1.61838     127.0.0.1.5000      ESTABLISHED
tcp4      0      0  127.0.0.1.5000      *.*                  LISTEN
[harsh@ragnarok ~/compiled]$ procstat -k $(pgrep -f exchange_server)
PID    TID  COMM          TDNAME          KSTACK
1925 100328 exchange_server -                mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_
kevent_generic sys_kevent amd64_syscall fast_syscall_common
[harsh@ragnarok ~/compiled]$ ps -H -o pid,tid,%cpu,wchan,comm -p $(pgrep -f exchange_server)
PID    LWP  %CPU  WCHAN          COMMAND
1925 100328 0.0  kqread exchange_server
[harsh@ragnarok ~/compiled]$
```

Figure 9: System-level evidence: both connections are **ESTABLISHED** and owned by the server (fds 5 and 6); `procstat -k` shows the single server thread parked in `kqueue_scan/kqueue_kevent`; and `ps -H` reports one thread with `WCHAN = kqread`.

From Figs. 8–9, three facts follow:

- Client 2’s round trip took **5 ms**; both clients are simultaneously **ESTABLISHED** on fds 5 and 6 of the same process.
- The server has **one thread** (TID 100328) – no dedicated thread hides the silent client.
- That thread is blocked in `kqueue_scan` (`WCHAN=kqread`), not in `recv()` on fd 5. All `Recv-Q` values are 0, so Client 1’s partial bytes have been read into the server’s own buffer and are waiting there for the missing `'\n'` – the wait is in user space, on a delimiter. Correspondingly, the server’s log line 6: `LOGIN active_client` exists for Client 2 but has no counterpart for fd 5.

Answer. Yes. Client 2 is accepted and answered OK in 0.005 s while Client 1 sits silently mid-message. The deciding operation is `kevent()` on the `kqueue` – the only blocking call in the loop – and `procstat -k/ps` confirm the single thread is parked in `kqueue_scan/kqread`. Because the server only touches descriptors already declared ready, an idle or half-speaking client costs nothing.

7 Experiment 5 – Multiple Clients and I/O Multiplexing

Approach and tools

The harness opens five simultaneous connections; only clients 1, 3 and 5 send an application message. Readiness is captured from two angles:

```
# 1. record every syscall the server makes, including kevent return values
$ ktrace -f /tmp/exp5.ktr -t cit -p $(pgrep -n exchange_server)
$ kdump -f /tmp/exp5.ktr | grep -E 'ident=[0-9]+, filter=EVFILT_READ, flags=0x20<EV_CLEAR>'

# 2. map fds back to sockets and see queued bytes
$ procstat -f $(pgrep -n exchange_server)
$ netstat -an -p tcp | grep 5000

# 3. second run: freeze the server as it starts listening so that the bytes
#     the clients send stay queued in the kernel and netstat can show them
$ kill -STOP $(pgrep -n exchange_server)
```


Observations

```

Started Exchange Server (PID 17538).
Listening on 127.0.0.1:5000
Exchange Server is listening.

Creating five simultaneous TCP connections...

Client 1: connected from 127.0.0.1:64427
Client 2: connected from 127.0.0.1:28067
Client 3: connected from 127.0.0.1:43792
Client 4: connected from 127.0.0.1:65363
Client 5: connected from 127.0.0.1:38472

Clients 1, 3, and 5 will send application messages.
Clients 2 and 4 will remain idle.

Client 1: sent application data.
5: LOGIN client_1
User client_1 logged in from 127.0.0.1:64427
Client 3: sent application data.
7: LOGIN client_3
User client_3 logged in from 127.0.0.1:43792
Client 5: sent application data.
9: LOGIN client_5
User client_5 logged in from 127.0.0.1:38472

The experiment is now in the observation phase.
Investigate which connections were active.
Press Ctrl-C when you are finished.
Stopping Exchange Server...
Client 127.0.0.1:64427 (user client_1) disconnected
Client 127.0.0.1:28067 disconnected
Client 127.0.0.1:43792 (user client_3) disconnected
Client 127.0.0.1:65363 disconnected
Client 127.0.0.1:38472 (user client_5) disconnected
Shutting down, closing 10 connection slots
Exchange server closed successfully
[harsh@ragnarok ~/compiled]$

```

Figure 10: Harness output for the traced run (server PID 17538). Clients 1, 3 and 5 connect from ports 64427, 43792 and 38472 and send LOGIN; clients 2 and 4 (ports 28067 and 65363) stay idle.

```

[harsh@ragnarok ~/compiled]$ rm -f /tmp/exp5.ktr
while ; do pid=$(pgrep -n exchange_server); if [ -n "$pid" ]; then ktrace -f /tmp/exp5.ktr -t cit -p $pid; echo "attached to $pid"; break; fi; done
attached to 17538
[harsh@ragnarok ~/compiled]$ ktrace -C
kdump -f /tmp/exp5.ktr | grep -B2 -A6 kevent | less
17538 exchange_server STRU struct kevent[] = { { ident=3, filter=EVFILT_READ, flags=0, fflags=0, data=0x1, udata=0x0 } }
17538 exchange_server RET kevent 1
17538 exchange_server CALL accept(0x3,0x820883d8c,0x820883db4)
17538 exchange_server RET accept -1 errno 53 Software caused connection abort
17538 exchange_server CALL accept(0x3,0x820883d8c,0x820883db4)
17538 exchange_server RET accept -1 errno 35 Resource temporarily unavailable
17538 exchange_server STRU struct kevent[] = { { ident=3, filter=EVFILT_READ, flags=0, fflags=0, data=0x5, udata=0x0 } }
17538 exchange_server RET kevent 1
17538 exchange_server CALL accept(0x3,0x820883d8c,0x820883db4)
17538 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:64427 }
17538 exchange_server RET accept 5
17538 exchange_server CALL fcntl(0x5,F_GETFL)
17538 exchange_server RET fcntl 6
17538 exchange_server CALL fcntl(0x5,F_SETFL,0x6<O_RDWR|O_NONBLOCK>)
17538 exchange_server RET fcntl 0
17538 exchange_server CALL kevent(0x4,0x820883cc0,0x2,0,0,0)
17538 exchange_server STRU struct kevent[] = { { ident=5, filter=EVFILT_READ, flags=0x21<EV_ADD|EV_CLEAR>, fflags=0, data=0, udata=0x0 } }
17538 exchange_server RET kevent 0
17538 exchange_server CALL accept(0x3,0x820883d8c,0x820883db4)
17538 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:28067 }
17538 exchange_server RET accept 6
17538 exchange_server CALL fcntl(0x6,F_GETFL)
17538 exchange_server RET fcntl 6
17538 exchange_server CALL fcntl(0x6,F_SETFL,0x6<O_RDWR|O_NONBLOCK>)
17538 exchange_server RET fcntl 0
17538 exchange_server CALL kevent(0x4,0x820883cc0,0x2,0,0,0)
17538 exchange_server STRU struct kevent[] = { { ident=6, filter=EVFILT_READ, flags=0x21<EV_ADD|EV_CLEAR>, fflags=0, data=0, udata=0x0 } }
17538 exchange_server RET kevent 0
17538 exchange_server CALL kevent(0x4,0x820883cc0,0x2,0,0,0)
17538 exchange_server STRU struct kevent[] = { { ident=6, filter=EVFILT_WRITE, flags=0x21<EV_ADD|EV_CLEAR>, fflags=0, data=0, udata=0x0 } }
17538 exchange_server STRU struct kevent[] = { }

```

Figure 11: Attaching `ktrace` to the server as soon as it starts, and the beginning of the trace: five `accept()` calls returning fds 5–9, each immediately set `O_NONBLOCK` and registered on the kqueue with `EVFILT_READ` and `EVFILT_WRITE`.

```

tcp4      0      0 127.0.0.1:5000      *.*      LISTEN
[harsh@ragnarok ~/compiled]$ kill $(pgrep -n exchange_server); netstat -an -p tcp | grep 5000
kill: usage: kill [-s sigspec | -n signum | -sigspec] pid | jobspec ... or kill -l [sigspec]
[harsh@ragnarok ~/compiled]$ PID=$(pgrep -n exchange_server); procstat -f $PID; ps -o pid,stat,wchan,comm -p $PID
PID COMM      FD T V FLAGS      REF      OFFSET PRO NAME
17538 exchange_server trace v r rw----- - - - /tmp/exp5.ktr
17538 exchange_server text v r r----- - - - /home/harsh/compiled/exchange_server
17538 exchange_server cwd v d r----- - - - /home/harsh/compiled
17538 exchange_server root v d r----- - - - /
17538 exchange_server 0 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 1 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 2 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 3 s - rw----- 2 0 TCP 0 0 127.0.0.1:5000 *:0
17538 exchange_server 4 k - rw----- 2 0 -
17538 exchange_server 5 s - rw----- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:64427
17538 exchange_server 6 s - rw----- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:28067
17538 exchange_server 7 s - rw----- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:43792
17538 exchange_server 8 s - rw----- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:65363
17538 exchange_server 9 s - rw----- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:38472
PID STAT WCHAN COMMAND
17538 Ss kqread exchange_server
[harsh@ragnarok ~/compiled]$ PID=$(pgrep -n exchange_server); procstat -f $PID; ps -o pid,stat,wchan,comm -p $PID
PID COMM      FD T V FLAGS      REF      OFFSET PRO NAME
17538 exchange_server trace v r rw----- - - - /tmp/exp5.ktr
17538 exchange_server text v r r----- - - - /home/harsh/compiled/exchange_server
17538 exchange_server cwd v d r----- - - - /home/harsh/compiled
17538 exchange_server root v d r----- - - - /
17538 exchange_server 0 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 1 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 2 v c rw----- 10 8713 - /dev/pts/1
17538 exchange_server 3 s - rw----- 2 0 TCP 0 0 127.0.0.1:5000 *:0
17538 exchange_server 4 k - rw----- 2 0 -
17538 exchange_server 5 s - rw----- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:64427
17538 exchange_server 6 s - rw----- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:28067
17538 exchange_server 7 s - rw----- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:43792
17538 exchange_server 8 s - rw----- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:65363
17538 exchange_server 9 s - rw----- 3 0 TCP 0 0 127.0.0.1:5000 127.0.0.1:38472
PID STAT WCHAN COMMAND

```

Figure 12: `procstat -f` for the server: fd 3 is the listening socket 127.0.0.1:5000 *:0, fd 4 is the kqueue, and fds 5–9 are the five client connections in the same order the harness created them.

```

17538 exchange_server GIO fd 5 read 15 bytes
"LOGIN client_1
"
17538 exchange_server RET read 15/0xf
17538 exchange_server CALL write(0x1,0x3bbbdd409000,0x12)
17538 exchange_server GIO fd 1 wrote 18 bytes
"5: LOGIN client_1
--
17538 exchange_server CALL kevent(0x4,0,0,0x820883df0,0x40,0)
17538 exchange_server STRU struct kevent[] = { }
17538 exchange_server STRU struct kevent[] = { { ident=7, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 } }
17538 exchange_server RET kevent 1
17538 exchange_server CALL read(0x7,0x3bbbdd4183c1,0xa0)
17538 exchange_server GIO fd 7 read 15 bytes
"LOGIN client_3
"
17538 exchange_server RET read 15/0xf
17538 exchange_server CALL write(0x1,0x3bbbdd409000,0x12)
17538 exchange_server GIO fd 1 wrote 18 bytes
"7: LOGIN client_3
--
17538 exchange_server CALL kevent(0x4,0,0,0x820883df0,0x40,0)
17538 exchange_server STRU struct kevent[] = { }
17538 exchange_server STRU struct kevent[] = { { ident=9, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 } }
17538 exchange_server RET kevent 1
17538 exchange_server CALL read(0x9,0x3bbbdd4186c1,0xa0)
17538 exchange_server GIO fd 9 read 15 bytes
"LOGIN client_5
"
17538 exchange_server RET read 15/0xf
17538 exchange_server CALL write(0x1,0x3bbbdd409000,0x12)
17538 exchange_server GIO fd 1 wrote 18 bytes
"9: LOGIN client_5
[harsh@ragnarok ~/compiled]$ kdump -f /tmp/exp5.ktr | grep -E 'ident=[0-9]+, filter=EVFILT_READ, flags=0x20<EV_CLEAR>\'
17538 exchange_server STRU struct kevent[] = { { ident=5, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 } }
17538 exchange_server STRU struct kevent[] = { { ident=7, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 } }
17538 exchange_server STRU struct kevent[] = { { ident=9, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 } }
[harsh@ragnarok ~/compiled]$

```

Figure 13: The decisive evidence: `kevent()` returns `EVFILT_READ` readiness for `ident=5`, `ident=7` and `ident=9` only, each with `data=0xf` (15 bytes pending), and the server reads exactly those descriptors. The final `grep` lists all three ready descriptors together.

The descriptor-to-client mapping from Figure 12 :

fd	Client	Peer port	Sent data
3	(listening socket)	*:0	—
4	(kqueue)	—	—
5	Client 1	64427	yes
6	Client 2	28067	no (idle)
7	Client 3	43792	yes
8	Client 4	65363	no (idle)
9	Client 5	38472	yes

The `kdump` trace (Fig. 13) shows `kevent()` returning `EVFILT_READ` exclusively for descriptors 5, 7, and 9, each with `data=0xf` (15 pending bytes, matching "LOGIN client_X\n") [cite: 2]. While

EVFILT_WRITE events fired for all sockets due to available send buffer space, only these EVFILT_READ events—drained until EAGAIN via EV_CLEAR—prove which clients actually sent data.

The same picture appears from outside the process. In a second run the server was stopped with `kill -STOP` before it called `accept()`; the three active clients' bytes remained queued in the kernel receive buffers, where `netstat` can show them.

```
[harsh@ragnarok ~/compiled]$ python3 experiment.py 5
=== Experiment 5: Multiple Clients and I/O Multiplexing (Optional) ===

Started Exchange Server (PID 9147).
Listening on 127.0.0.1:5000
Exchange Server is listening.

Creating five simultaneous TCP connections...

Client 1: connected from 127.0.0.1:42196
Client 2: connected from 127.0.0.1:62300
Client 3: connected from 127.0.0.1:13037
Client 4: connected from 127.0.0.1:26978
Client 5: connected from 127.0.0.1:53133

Clients 1, 3, and 5 will send application messages.
Clients 2 and 4 will remain idle.

Client 1: sent application data.
Client 3: sent application data.
Client 5: sent application data.

The experiment is now in the observation phase.
Investigate which connections were active.
Press Ctrl-C when you are finished.
Stopping Exchange Server...
Exchange Server did not terminate; killing it.
[harsh@ragnarok ~/compiled]$
```

Figure 14: The second run (server PID 9147): client 1 = 42196, client 2 = 62300, client 3 = 13037, client 4 = 26978, client 5 = 53133.

```
[harsh@ragnarok ~/compiled]$ netstat -an -p tcp | grep 5000
tcp4      15      0 127.0.0.1.5000      127.0.0.1.53133    ESTABLISHED
tcp4       0      0 127.0.0.1.53133    127.0.0.1.5000    ESTABLISHED
tcp4       0      0 127.0.0.1.5000     127.0.0.1.26978    ESTABLISHED
tcp4       0      0 127.0.0.1.26978    127.0.0.1.5000    ESTABLISHED
tcp4      15      0 127.0.0.1.5000     127.0.0.1.13037    ESTABLISHED
tcp4       0      0 127.0.0.1.13037    127.0.0.1.5000    ESTABLISHED
tcp4       0      0 127.0.0.1.5000     127.0.0.1.62300    ESTABLISHED
tcp4       0      0 127.0.0.1.62300    127.0.0.1.5000    ESTABLISHED
tcp4      15      0 127.0.0.1.5000     127.0.0.1.42196    ESTABLISHED
tcp4       0      0 127.0.0.1.42196    127.0.0.1.5000    ESTABLISHED
tcp4       0      0 127.0.0.1.5000     127.0.0.1.59941    CLOSED
tcp4       0      0 127.0.0.1.5000     *.*                LISTEN
[harsh@ragnarok ~/compiled]$
```

Figure 15: `netstat -an -p tcp`: the server-side rows for ports 53133, 13037 and 42196 (clients 5, 3 and 1) carry `Recv-Q = 15`; the rows for the idle clients 62300 and 26978 carry `Recv-Q = 0`.

Answer. Exactly three of the five connections are ready – clients 1, 3, and 5. `kevent()` returns `EVFILT_READ` for `ident=5, 7, and 9` with `data=15`, mapped by `procstat -f` to peer ports 64427/43792/38472; `netstat` independently shows `Recv-Q=15` on the same three sockets and `Recv-Q=0` on the two idle ones.

8 Experiment 6 – FIN vs. RST: Orderly and Abrupt Termination

Approach and tools

The harness runs two parts back to back on one server. **Part A:** an orderly close via `shutdown(SHUT_WR)`. **Part B:** an abortive close via `SO_LINGER=0` then `close()`. Three observers run in parallel:

```
$ tcpdump -i lo0 -n -tttt -S tcp port 5000 # packets
$ while :; do date +%H:%M:%S; netstat -an -p tcp | grep 5000; sleep 1; done # states
$ ktrace -f /tmp/exp6.ktr -t cit -p $(pgrep -n exchange_server) # syscalls
$ kdump -f /tmp/exp6.ktr | grep -E 'EV_EOF|accept|close'
```

Observations

```
root@agnarok:/home/harsh/compiled # tcpdump -i lo0 -n -tttt -S tcp port 5000
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
2026-09-08 15:11:11.480199 IP 127.0.0.1.23897 > 127.0.0.1.5000: Flags [S], seq 3215378964, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 56364
4169 ecr 0], length 0
2026-09-08 15:11:11.480553 IP 127.0.0.1.5000 > 127.0.0.1.23897: Flags [R.], seq 0, ack 3215378965, win 0, length 0
2026-09-08 15:11:11.588739 IP 127.0.0.1.56301 > 127.0.0.1.5000: Flags [S], seq 4201627391, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 27492
92607 ecr 0], length 0
2026-09-08 15:11:11.590037 IP 127.0.0.1.5000 > 127.0.0.1.56301: Flags [S.], seq 3093378638, ack 4201627392, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 649293137 ecr 2749292607], length 0
2026-09-08 15:11:11.590111 IP 127.0.0.1.56301 > 127.0.0.1.5000: Flags [.], ack 3093378639, win 1277, options [nop,nop,TS val 2749292618 ecr 649293137], length 0
2026-09-08 15:11:11.590489 IP 127.0.0.1.56301 > 127.0.0.1.5000: Flags [R.], seq 4201627392, ack 3093378639, win 0, options [nop,nop,TS val 2749292618 ecr 649293137], length 0
2026-09-08 15:11:11.593527 IP 127.0.0.1.56436 > 127.0.0.1.5000: Flags [S], seq 751935905, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 332611
7646 ecr 0], length 0
2026-09-08 15:11:11.593600 IP 127.0.0.1.5000 > 127.0.0.1.56436: Flags [S.], seq 3975613210, ack 751935906, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 798935012 ecr 3326117646], length 0
2026-09-08 15:11:11.593676 IP 127.0.0.1.56436 > 127.0.0.1.5000: Flags [.], ack 3975613211, win 1277, options [nop,nop,TS val 3326117646 ecr 798935012], length 0
2026-09-08 15:11:11.593872 IP 127.0.0.1.56436 > 127.0.0.1.5000: Flags [F.], seq 751935906, ack 3975613211, win 1277, options [nop,nop,TS val 3326117646 ecr 798935012], length 0
2026-09-08 15:11:11.593937 IP 127.0.0.1.5000 > 127.0.0.1.56436: Flags [.], ack 751935907, win 1277, options [nop,nop,TS val 798935012 ecr 3326117646], length 0
2026-09-08 15:11:11.607184 IP 127.0.0.1.5000 > 127.0.0.1.56436: Flags [F.], seq 3975613211, ack 751935907, win 1277, options [nop,nop,TS val 798935022 ecr 3326117646], length 0
2026-09-08 15:11:11.607269 IP 127.0.0.1.56436 > 127.0.0.1.5000: Flags [.], ack 3975613212, win 1277, options [nop,nop,TS val 3326117656 ecr 798935022], length 0
2026-09-08 15:11:21.623193 IP 127.0.0.1.38885 > 127.0.0.1.5000: Flags [S], seq 2754814503, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 42037
40687 ecr 0], length 0
2026-09-08 15:11:21.623253 IP 127.0.0.1.5000 > 127.0.0.1.38885: Flags [S.], seq 243721720, ack 2754814504, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 1483227215 ecr 4203740687], length 0
2026-09-08 15:11:21.623307 IP 127.0.0.1.38885 > 127.0.0.1.5000: Flags [.], ack 243721721, win 1277, options [nop,nop,TS val 4203740687 ecr 1483227215], length 0
2026-09-08 15:11:21.625160 IP 127.0.0.1.38885 > 127.0.0.1.5000: Flags [R.], seq 2754814504, ack 243721721, win 0, options [nop,nop,TS val 4203740687 ecr 1483227215], length 0
^C
17 packets captured
```

Figure 16: tcpdump on lo0 covering both parts. Part A (port 56436, 15:11:11) ends with a four-packet FIN exchange; Part B (port 38885, 15:11:21) ends with a single RST about 2 ms after its own SYN.

```
---
15:11:12
tcp4      0      0 127.0.0.1.56436 127.0.0.1.5000  CLOSED
tcp4      0      0 127.0.0.1.5000 *.*          LISTEN
---
15:11:13
tcp4      0      0 127.0.0.1.56436 127.0.0.1.5000  CLOSED
tcp4      0      0 127.0.0.1.5000 *.*          LISTEN
---
15:11:14
tcp4      0      0 127.0.0.1.56436 127.0.0.1.5000  CLOSED
tcp4      0      0 127.0.0.1.5000 *.*          LISTEN
---
15:11:15
tcp4      0      0 127.0.0.1.56436 127.0.0.1.5000  CLOSED
tcp4      0      0 127.0.0.1.5000 *.*          LISTEN
---
15:11:16
tcp4      0      0 127.0.0.1.56436 127.0.0.1.5000  CLOSED
tcp4      0      0 127.0.0.1.5000 *.*          LISTEN
---
15:11:17
tcp4      0      0 127.0.0.1.56436 127.0.0.1.5000  CLOSED
tcp4      0      0 127.0.0.1.5000 *.*          LISTEN
---
15:11:18
tcp4      0      0 127.0.0.1.56436 127.0.0.1.5000  CLOSED
tcp4      0      0 127.0.0.1.5000 *.*          LISTEN
---
15:11:19
tcp4      0      0 127.0.0.1.56436 127.0.0.1.5000  CLOSED
tcp4      0      0 127.0.0.1.5000 *.*          LISTEN
---
15:11:20
tcp4      0      0 127.0.0.1.56436 127.0.0.1.5000  CLOSED
tcp4      0      0 127.0.0.1.5000 *.*          LISTEN
---
15:11:21
tcp4      0      0 127.0.0.1.56436 127.0.0.1.5000  CLOSED
tcp4      0      0 127.0.0.1.5000 *.*          LISTEN
---
```

Figure 17: After the orderly close (Part A): from 15:11:12 to 15:11:21 the connection remains listed once per second, in state CLOSED, because the client's descriptor is still open.

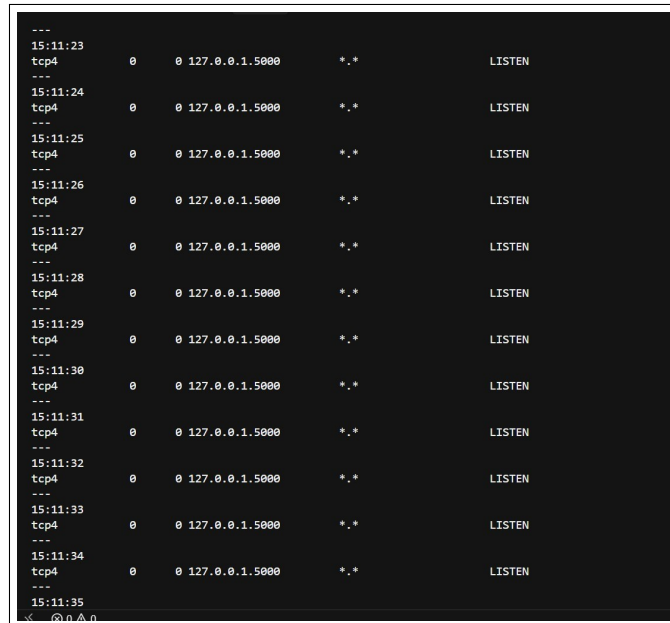


Figure 18: After the abortive close (Part B): from 15:11:23 onwards only the LISTEN socket is left. The reset connection was destroyed immediately and never appears in a single snapshot.

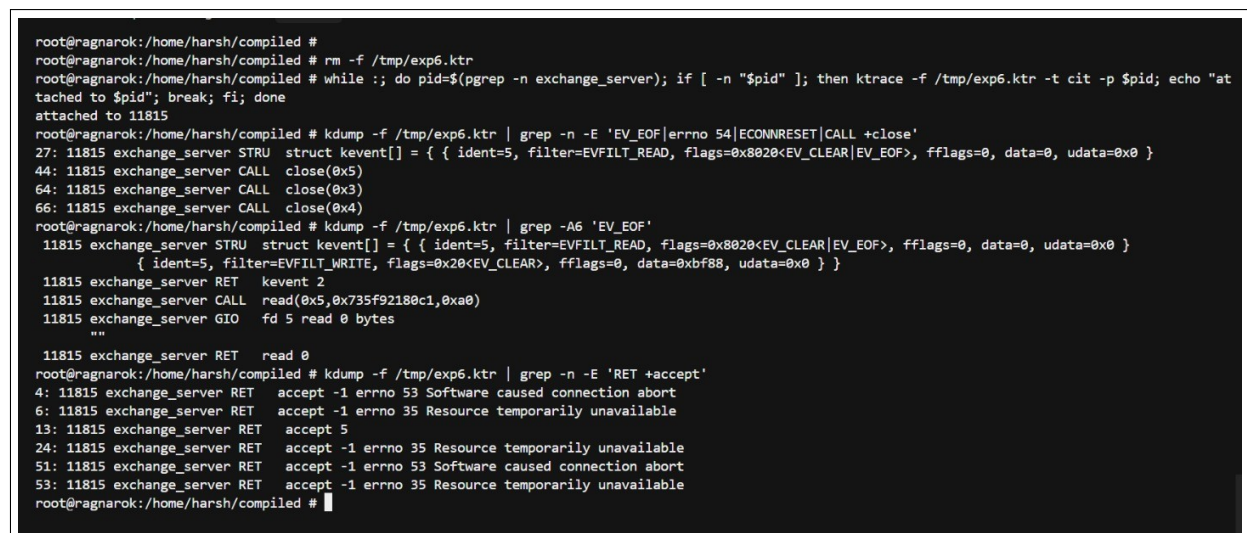


Figure 19: Server-side syscalls. The orderly close surfaces as EVFILT_READ with EV_EOF and read() returning 0 bytes; the abortive one never becomes a connection at all – accept() fails with errno 53, Software caused connection abort. No ECONNRESET appears anywhere in the trace.

Contrast:

	Orderly (Part A)	Abrupt (Part B)
Trigger	shutdown(SHUT_WR) / close()	SO_LINGER=0 then close()
Packets	FIN → ACK → FIN → ACK (4)	a single RST (1)
Data already sent	delivered before the FIN is reported	discarded
Peer's socket	becomes readable, read() returns 0	destroyed; a later read/write gives ECONNRESET/EPIPE
netstat	visible as CLOSED while the fd lives (Fig. 17)	gone instantly (Fig. 18)
Our server's reaction	EV_EOF, deregister filters, log, close()	accept() returns ECONNABORTED, skipped

Answer. Orderly shutdown is a four-packet FIN/ACK exchange that guarantees delivery of previously sent bytes; the server sees `EVFILT_READ+EV_EOF` and `read()=0`. Abrupt termination is a single RST: the connection is torn down unilaterally, any queued data is discarded, and the entry vanishes from `netstat` immediately – here surfacing as `accept()=ECONNABORTED` because the reset arrived pre-accept (post-accept it would give `EPIPE/ECONNRESET`).

9 Experiment 7 – Backpressure and the Slow Receiver

Approach and tools

Four clients are connected – a draining Market-Data client, a non-reading (slow) Market-Data client, and two Traders – while 5000 BUY/SELL matches push 5000 TRADE messages (17 B each) to both subscribers. Socket buffers were shrunk to 8 KiB with auto-tuning disabled to force backpressure quickly.

```
$ sysctl net.inet.tcp.recvbuf_auto=0 net.inet.tcp.sendbuf_auto=0
$ sysctl net.inet.tcp.recvspace=8192 net.inet.tcp.sendspace=8192
$ tcpdump -i lo0 -n -tttt "tcp port 5000 and tcp[14:2] = 0"      # zero-window
$ while :; do date +%H:%M:%S; netstat -an -p tcp | grep 5000; sleep 2; done    # queues
$ python3 experiment.py 7
$ while :; do ps -o pid,rss,vsz,%cpu,stat,wchan -p $PID; sleep 2; done        # server
$ diff <(netstat -s -p tcp <before>) <(netstat -s -p tcp <after>)            # TCP counters
```

Observations

```
root@ragnarok:/home/harsh/compiled #
root@ragnarok:/home/harsh/compiled # grep -E 'connected from|Generated' /tmp/exp7/30_harness.txt
Normal Market-Data Client: connected from 127.0.0.1:35877
Slow Market-Data Client: connected from 127.0.0.1:50266
Buyer Trader: connected from 127.0.0.1:65121
Seller Trader: connected from 127.0.0.1:49148
Generated 5000 matching trades.
root@ragnarok:/home/harsh/compiled #
```

Figure 20: The four connections and the generated load: normal Market-Data client on port 35877, slow Market-Data client on port 50266, buyer on 65121, seller on 49148, and **5000 matching trades** generated.

```
root@ragnarok:/home/harsh/compiled # cat /tmp/exp7/00_sysctl.txt
net.inet.tcp.recvspace: 8192
net.inet.tcp.sendspace: 8192
net.inet.tcp.recvbuf_auto: 0
net.inet.tcp.sendbuf_auto: 0
root@ragnarok:/home/harsh/compiled #
```

Figure 21: Socket-buffer parameters recorded at the start of the run.

```

root@ragnarok:/home/harsh/compiled #
root@ragnarok:/home/harsh/compiled # LINE=$(grep -n '85000' /tmp/exp7/20_netstat.txt | head -1 | cut -d: -f1)
root@ragnarok:/home/harsh/compiled # sed -n "${((LINE-7))},${((LINE+7))}p" /tmp/exp7/20_netstat.txt
---
16:49:34
tcp4      0      0 127.0.0.1.5000    127.0.0.1.49148  ESTABLISHED
tcp4      16      0 127.0.0.1.49148  127.0.0.1.5000  ESTABLISHED
tcp4      0      0 127.0.0.1.5000    127.0.0.1.65121  ESTABLISHED
tcp4      18      0 127.0.0.1.65121  127.0.0.1.5000  ESTABLISHED
tcp4      0      0 127.0.0.1.5000    127.0.0.1.50266  ESTABLISHED
tcp4      85000   0 127.0.0.1.50266  127.0.0.1.5000  ESTABLISHED
tcp4      0      0 127.0.0.1.5000    127.0.0.1.35877  ESTABLISHED
tcp4      0      0 127.0.0.1.35877  127.0.0.1.5000  ESTABLISHED
tcp4      0      0 127.0.0.1.5000    *.*              LISTEN
---
16:49:36
tcp4      0      0 127.0.0.1.5000    127.0.0.1.49148  ESTABLISHED
tcp4      16      0 127.0.0.1.49148  127.0.0.1.5000  ESTABLISHED
root@ragnarok:/home/harsh/compiled #

```

Figure 22: Queue depths at the peak. The slow client’s own socket (127.0.0.1.50266) holds Recv-Q = 85000 bytes that its application has never read, while the server’s send queues to *all* clients, including the slow one, are essentially empty.

```

root@ragnarok:/home/harsh/compiled #
root@ragnarok:/home/harsh/compiled # head -3 /tmp/exp7/40_server.txt; echo ...; tail -4 /tmp/exp7/40_server.txt
16:46:10
  PID  RSS   VSZ %CPU STAT WCHAN
19612 2180 13748  1.1  Ss kqread
...
16:49:47
  PID  RSS   VSZ %CPU STAT WCHAN
19612 2180 13748  0.0  Ss kqread
---
root@ragnarok:/home/harsh/compiled #

```

Figure 23: The server (PID 19612) at the beginning and the end of the run: RSS unchanged at 2180 KiB, CPU falling back to 0.0%, and WCHAN still kqread.

```

root@ragnarok:/home/harsh/compiled #
root@ragnarok:/home/harsh/compiled # grep -c 'win 0' /tmp/exp7/10_zerowindow.txt
2
root@ragnarok:/home/harsh/compiled # cat /tmp/exp7/10_zerowindow.txt
2026-09-08 16:46:08.760257 IP 127.0.0.1.5000 > 127.0.0.1.27309: Flags [R.], seq 0, ack 539677606, win 0, length 0
2026-09-08 16:46:08.866078 IP 127.0.0.1.53893 > 127.0.0.1.5000: Flags [R.], seq 1531973612, ack 446783982, win 0, options [nop,nop,TS val 1087438920 ecr 2746475297], length 0
root@ragnarok:/home/harsh/compiled #

```

Figure 24: Searching the capture for zero-window advertisements. The only two matches are RST segments (which always carry win 0); no genuine zero-window advertisement was ever sent.

Across 108 `netstat` samples the slow client’s Recv-Q rises monotonically from 476 B to exactly $5000 \times 17 = 85\,000$ B (Fig. 22), while the server’s Send-Q toward it never exceeds 34 B. No zero window appears (0 window probes / 0 persist timeouts; Fig. 24). The server stays in `kqread` with RSS fixed at 2180 KiB and CPU falling to 0.0% (Fig. 23); all 5000 trades still complete for the other clients.

The 85 KiB plateau exceeds the nominal 8 KiB `recvspace` because the hard ceiling is `kern.ipc.maxsockbuf`, so the receive buffer was never exhausted and TCP flow control never engaged. Had it been, the non-blocking path would be `win 0` → `Send-Q full` → `write()=EAGAIN` → per-connection outbuf drained on `EVFILT_WRITE` – still no global stall. A blocking `write()` would park in `sbwait` and hold every client.

Answer. Unread data piles up only on the slow client (Recv-Q→85 000 B); Send-Q≤34 B and no zero window is advertised. Other clients finish all 5000 trades; the server never leaves `kqread`. Non-blocking writes plus per-connection buffers confine backpressure to the slow reader.

10 Experiment 8 – Unexpected Client Disconnection

Approach and tools

The harness starts a separate Market-Data Client that subscribes to JNST and then sleeps. After 20 trades it is SIGKILLED – the application runs no shutdown code. A second Market-Data Client and two Traders continue for the comparison. Packets, socket states, and server syscalls are recorded:

```
$ tcpdump -i lo0 -n -tttt -S tcp port 5000
$ while ;; do date +%H:%M:%S; netstat -an -p tcp | grep 5000; sleep 1; done
$ ktrace -f /tmp/exp8/server.ktr -t cit -p $(pgrep -n exchange_server)
$ kdump -f /tmp/exp8/server.ktr | grep -B1 -E 'RET +accept [0-9]'
```

Observations

```
root@ragnarok:/home/harsh/compiled # grep 'connected from' /tmp/exp8/30_harness.txt
Surviving Market-Data Client: connected from 127.0.0.1:10123
Buyer Trader: connected from 127.0.0.1:38997
Seller Trader: connected from 127.0.0.1:26991
root@ragnarok:/home/harsh/compiled #
```

Figure 25: The three connections created directly by the harness: surviving Market-Data client on port 10123, buyer on 38997, seller on 26991. The client that will be killed is a separate process and connects on its own.

```
root@ragnarok:/home/harsh/compiled # kdump -f /tmp/exp8/server.ktr | grep -B1 -E 'RET +accept [0-9]'
31814 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:10123 }
31814 exchange_server RET  accept 5
--
31814 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:38997 }
31814 exchange_server RET  accept 6
--
31814 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:26991 }
31814 exchange_server RET  accept 7
--
31814 exchange_server STRU struct sockaddr { AF_INET, 127.0.0.1:48664 }
31814 exchange_server RET  accept 8
root@ragnarok:/home/harsh/compiled #
```

Figure 26: kdump of the server's accept() calls identifies the fourth connection: fd 5 = 10123 (surviving MD), fd 6 = 38997 (buyer), fd 7 = 26991 (seller), fd 8 = 48664 (the client that will disappear).


```

root@ragnarok:/home/harsh/compiled # awk '/^17:10:02$/,/^17:10:06$/ ' /tmp/exp8/20_netstat.txt
17:10:02
tcp4      0      0 127.0.0.1.5000      127.0.0.1.48664    ESTABLISHED
tcp4    136      0 127.0.0.1.48664      127.0.0.1.5000      ESTABLISHED
tcp4      0     16 127.0.0.1.5000      127.0.0.1.26991     ESTABLISHED
tcp4     16      0 127.0.0.1.26991      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.38997     ESTABLISHED
tcp4     18      0 127.0.0.1.38997      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.10123     ESTABLISHED
tcp4      0      0 127.0.0.1.10123      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
17:10:03
tcp4      0      0 127.0.0.1.5000      127.0.0.1.48664    ESTABLISHED
tcp4    255      0 127.0.0.1.48664      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.26991     ESTABLISHED
tcp4     16      0 127.0.0.1.26991      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.38997     ESTABLISHED
tcp4     18      0 127.0.0.1.38997      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.10123     ESTABLISHED
tcp4      0      0 127.0.0.1.10123      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
17:10:04
tcp4      0     34 127.0.0.1.5000      127.0.0.1.26991     ESTABLISHED
tcp4      0      0 127.0.0.1.26991      127.0.0.1.5000      ESTABLISHED
tcp4      0     36 127.0.0.1.5000      127.0.0.1.38997     ESTABLISHED
tcp4      0      0 127.0.0.1.38997      127.0.0.1.5000      ESTABLISHED
tcp4      0     17 127.0.0.1.5000      127.0.0.1.10123     ESTABLISHED
tcp4     17      0 127.0.0.1.10123      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
---
17:10:05
tcp4      0     34 127.0.0.1.5000      127.0.0.1.26991     ESTABLISHED
tcp4      0      0 127.0.0.1.26991      127.0.0.1.5000      ESTABLISHED
tcp4      0     36 127.0.0.1.5000      127.0.0.1.38997     ESTABLISHED
tcp4      0      0 127.0.0.1.38997      127.0.0.1.5000      ESTABLISHED
tcp4      0     17 127.0.0.1.5000      127.0.0.1.10123     ESTABLISHED

```

Figure 27: netstat one second at a time. At 17:10:02 and 17:10:03 the doomed connection (127.0.0.1.48664) is ESTABLISHED with 136 and then 255 bytes of unread market data in its Recv-Q. At 17:10:04 it is gone completely, while the other three connections carry on exchanging data.

```

root@ragnarok:/home/harsh/compiled # grep -E 'Flags \[R\|F\]' /tmp/exp8/10_tcpdump.txt
2026-09-08 17:09:59.757302 IP 127.0.0.1.5000 > 127.0.0.1.27571: Flags [R.], seq 0, ack 4090159394, win 0, length 0
2026-09-08 17:09:59.862060 IP 127.0.0.1.59148 > 127.0.0.1.5000: Flags [R.], seq 1528691534, ack 2433209448, win 0, options [nop,nop,TS val 1409265845 ecr 158687807], length 0
2026-09-08 17:10:03.772273 IP 127.0.0.1.48664 > 127.0.0.1.5000: Flags [F.], seq 584026206, ack 2245618379, win 1277, options [nop,nop,TS val 3883213896 ecr 700284551], length 0
2026-09-08 17:10:03.776622 IP 127.0.0.1.5000 > 127.0.0.1.48664: Flags [F.], seq 2245618379, ack 584026207, win 1277, options [nop,nop,TS val 700284692 ecr 3883213896], length 0
2026-09-08 17:10:23.524663 IP 127.0.0.1.10123 > 127.0.0.1.5000: Flags [F.], seq 676359951, ack 3413299175, win 1277, options [nop,nop,TS val 823780914 ecr 152491013], length 0
2026-09-08 17:10:23.524712 IP 127.0.0.1.38997 > 127.0.0.1.5000: Flags [F.], seq 2755748733, ack 3346130460, win 1277, options [nop,nop,TS val 827532680 ecr 3480140277], length 0
2026-09-08 17:10:23.524739 IP 127.0.0.1.26991 > 127.0.0.1.5000: Flags [F.], seq 3239146630, ack 3501322117, win 1277, options [nop,nop,TS val 2552839797 ecr 1252052305], length 0
2026-09-08 17:10:23.525699 IP 127.0.0.1.5000 > 127.0.0.1.10123: Flags [F.], seq 3413299175, ack 676359952, win 1277, options [nop,nop,TS val 152506135 ecr 823780914], length 0
2026-09-08 17:10:23.526754 IP 127.0.0.1.5000 > 127.0.0.1.38997: Flags [F.], seq 3346130460, ack 2755748734, win 1277, options [nop,nop,TS val 3480155361 ecr 827532680], length 0
2026-09-08 17:10:23.527424 IP 127.0.0.1.5000 > 127.0.0.1.26991: Flags [F.], seq 3501322117, ack 3239146631, win 1277, options [nop,nop,TS val 1252067389 ecr 2552839797], length 0
root@ragnarok:/home/harsh/compiled #

```

Figure 28: Every FIN and RST on the port. At 17:10:03.772273 the killed client's kernel sends a FIN, the server ACKs it and replies with its own FIN 4 ms later. The other three connections only close at 17:10:23, when the experiment ends.

```

root@ragnarok:/home/harsh/compiled # kdump -f /tmp/exp8/server.ktr | sed -n '1620,1650p'
31814 exchange_server CALL kevent(0x4,0,0,0x82085e0c0,0x40,0)
31814 exchange_server STRU struct kevent[] = { }
31814 exchange_server STRU struct kevent[] = { { ident=6, filter=EVFILT_WRITE, flags=0x20<EV_CLEAR>, fflags=0, data=0xbf88, udata=0x0 }
{ ident=7, filter=EVFILT_WRITE, flags=0x20<EV_CLEAR>, fflags=0, data=0xbf88, udata=0x0 } }
31814 exchange_server RET kevent 2
31814 exchange_server CALL kevent(0x4,0,0,0x82085e0c0,0x40,0)
31814 exchange_server STRU struct kevent[] = { }
31814 exchange_server STRU struct kevent[] = { { ident=8, filter=EVFILT_READ, flags=0x8020<EV_CLEAR|EV_EOF>, fflags=0, data=0, udata=0x0 } }
31814 exchange_server RET kevent 1
31814 exchange_server CALL read(0x8,0x3172b7018541,0xa0)
31814 exchange_server GIO fd 8 read 0 bytes
""
31814 exchange_server RET read 0
31814 exchange_server CALL kevent(0x4,0x82085e000,0x2,0,0,0)
31814 exchange_server STRU struct kevent[] = { { ident=8, filter=EVFILT_READ, flags=0x2<EV_DELETE>, fflags=0, data=0, udata=0x0 }
{ ident=8, filter=EVFILT_WRITE, flags=0x2<EV_DELETE>, fflags=0, data=0, udata=0x0 } }
31814 exchange_server STRU struct kevent[] = { }
31814 exchange_server RET kevent 0
31814 exchange_server CALL write(0x1,0x3172b7009000,0x24)
31814 exchange_server GIO fd 1 wrote 36 bytes
"Client 127.0.0.1:48664 disconnected"
"
31814 exchange_server RET write 36/0x24
31814 exchange_server CALL close(0x8)
31814 exchange_server RET close 0
31814 exchange_server CALL kevent(0x4,0,0,0x82085e0c0,0x40,0)
31814 exchange_server STRU struct kevent[] = { }
31814 exchange_server STRU struct kevent[] = { { ident=6, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0xf, udata=0x0 }
{ ident=7, filter=EVFILT_READ, flags=0x20<EV_CLEAR>, fflags=0, data=0x10, udata=0x0 } }
31814 exchange_server RET kevent 2
31814 exchange_server CALL read(0x6,0x3172b7018241,0xa0)
root@ragnarok:/home/harsh/compiled #

```

Figure 29: The server’s view of the death: `kevent()` reports `ident=8` with `EV_CLEAR|EV_EOF`, `read(8)` returns **0 bytes**, the filters are deleted, the disconnect is logged and `close(8)` is called. The very next `kevent()` already returns readiness for `ident=6` and `ident=7` – the other clients were never interrupted.

From Figs. 27–29, three observations follow:

- **Kernel closes, not the application.** `SIGKILL` runs no shutdown code; FreeBSD releases the FDs on process exit and the TCP stack performs a normal active close. On the wire a killed process therefore looks like an orderly close: `FIN`, not `RST`.
- **Unread data is visible up to the moment of death.** `Recv-Q` on the doomed socket grows `17 → 136 → 255 B`; one second later the socket is gone entirely with no `TIME_WAIT` row (same `net.inet.tcp.nolocaltimewait` behaviour as Experiment 2).
- **Detection is immediate and local.** No timeout, keep-alive or failed write was needed: the `FIN` made the descriptor readable, `read()` returned 0, and the slot was released. `EV_EOF` again carried `fflags=0`; the trace contains no `EPIPE` or `ECONNRESET`. The next `kevent()` was already serving fds 6 and 7, and the surviving subscriber received all 50 post-kill trades.

Harder failure modes. Killing a process still leaves the OS to send the `FIN`. If the machine died or the network were cut, no packet would arrive and the socket would remain `ESTABLISHED` indefinitely; detection would then require a write (giving `ETIMEDOUT` or `ECONNRESET`) or `SO_KEEPALIVE` probes.

Answer. Although the client process ran no shutdown code, the FreeBSD kernel closed its descriptors on exit, producing a normal `FIN` exchange (not a reset); the socket vanished from `netstat` within a second, having held 255 B of undelivered data at the moment of death. The server detected the failure via `EVFILT_READ+EV_EOF` with `read()=0`, then deregistered the filters and closed fd 8; the next `kevent()` was already serving fds 6 and 7.

11 Summary of Tools Used

Tool	Experiments	What it contributed
<code>sockstat -4 -l / -c</code>	1, 4	Maps every socket to its owning process and file descriptor; separates listening from connected sockets.
<code>netstat -an -p tcp</code>	1, 2, 4, 5, 6, 7, 8	TCP state column, and the <code>Recv-Q/Send-Q</code> depths used as evidence that a partial message had already been consumed by the server (Exp. 4), of readiness (Exp. 5) and of backpressure (Exp. 7, 8).
<code>netstat -s -p tcp</code>	7	Protocol counters: 0 window probes and 0 persist timeouts proved no zero-window event occurred.
<code>tcpdump -i lo0</code>	2, 3, 6, 7, 8	The packet-level ground truth: handshake, per-segment framing (<code>-A</code>), FIN vs. RST, zero-window search.
<code>ktrace + kdump</code>	5, 6, 8	The server's own syscalls: <code>kevent()</code> return values naming the ready descriptors, <code>read()</code> returning 0 at EOF, <code>accept()</code> <code>errno</code> values.
<code>procstat -k</code>	4	Kernel stack of the server thread – proved it sleeps in <code>kqueue_scan</code> , not in <code>recv()</code> .
<code>procstat -f</code>	5	Descriptor table, used to map <code>kevent</code> ids back to real client sockets.
<code>ps -H -o ...,wchan</code>	4, 7	Thread count, <code>WCHAN</code> (<code>kqread</code>), RSS and CPU of the server under load.
<code>sysctl</code>	7	Socket-buffer parameters that gate backpressure.

A Bonus – Connection Scalability and I/O Design

This appendix reports the results of Section 6.9 of the handout: driving the Exchange Server up to $N = 70\,000$ simultaneous idle TCP connections, measuring the per- N resource footprint, and analysing the bottlenecks reached. All measurements were taken on the same FreeBSD 14.4-RELEASE VM used for Experiments 1–8.

A.1 Setup and tooling

Two small helper programs, both in `bonus/`, were written for this section. The Exchange Server, Trader Client and Market-Data Client themselves are otherwise unchanged from Experiments 1–8 (with the single exception noted in §A.2).

`bonus/idle_clients.py` – client generator. Opens N TCP connections to the Exchange Server and holds them `ESTABLISHED` without ever sending or receiving application data. It raises `RLIMIT_NOFILE` to $N + 128$ before starting, reports progress and connect rate every `-batch` connections, and records the index and `errno` of every `connect()` failure so the count at which the server first fails is unambiguous. Three additive optional flags (`--src-ips`, `--src-cidr`, `--src-count`) enable source-IP rotation and are described in §A.3; without them the script behaves exactly like a default client (kernel-picked `127.0.0.1` + ephemeral port).

```
$ python3 bonus/idle_clients.py 127.0.0.1 5000 50000 --batch 2000
$ python3 bonus/idle_clients.py 127.0.0.1 5000 70000 --batch 2000 \
  --src-cidr 127.0.0.0/8 --src-count 4
```

`bonus/measure.sh` – one snapshot per row of the table. Runs while `idle_clients.py` is holding its connections open and captures every column of the required Section 6.9 table in a single output:

```
$ ./bonus/measure.sh <server-pid> 5000

established conns  netstat -an -p tcp | awk ... | wc -l (+ sockstat cross-check)
server FDs        procstat -f <pid> | tail -n +2 | wc -l
memory / CPU      ps -o pid,rss,vsz,%cpu,%mem,nlwp,command -p <pid>
system-wide FDs   sysctl kern.openfiles kern.maxfiles kern.maxfilesperproc
socket buffers     netstat -m
                  sysctl kern.ipc.maxsockbuf net.inet.tcp.sendspace \
                  net.inet.tcp.recvspace kern.ipc.somaxconn
I/O design check  procstat -k <pid>
```

OS tuning applied. FreeBSD’s defaults (~ 64 k sockets, ~ 29 k FDs per proc, `SOMAXCONN=128`) cannot support 70 k connections. The tunables raised before the runs are summarised in Table 1.

Parameter	Scope	Default	Set to	Reason
<code>kern.ipc.maxsockets</code>	<code>loader.conf</code>	64 177	400 000	Kernel socket-struct pool; loopback burns 2/conn, so default caps at ~ 32 k conns.
<code>kern.ipc.nmbclusters</code>	<code>loader.conf</code>	125 007	500 000	mbuf clusters back socket buffers; exhaustion returns <code>ENOBUFFS</code> from <code>socket()</code> .
<code>kern.ipc.somaxconn</code>	<code>sysctl.conf</code>	128	65 535	Listen-backlog ceiling; prevents SYN drops under connect flood.
<code>kern.maxfiles</code>	<code>sysctl.conf</code>	29 053	300 000	System-wide FD cap; 70 k loopback conns need ~ 140 k FDs.
<code>kern.maxfilesperproc</code>	<code>sysctl.conf</code>	26 148	250 000	Per-process FD cap; server and generator each hold ~ 70 k FDs.
<code>portrange.first</code>	<code>sysctl.conf</code>	49 152	10 000	Widens ephemeral pool from ~ 16 k to ~ 55 k source ports.
<code>portrange.last</code>	<code>sysctl.conf</code>	65 535	65 535	(unchanged upper bound)
<code>RLIMIT_NOFILE</code>	per shell	57 753	250 000	Inherited soft FD cap; raised before launching server / generator.
lo0 aliases	<code>ifconfig</code>	127.0.0.1/8	+127.0.0.{2,3,4}/32	Only for $N > 55\,000$; needed for source-IP rotation on the generator.

Table 1: OS-level parameters altered for the bonus experiment (FreeBSD 14.4-RELEASE). `portrange.* = net.inet.ip.portrange.*`.

Server-imposed vs. OS-imposed limitations. Per the requirement to distinguish the two, the parameters in Table 1 are strictly *OS/VM defaults*: they allocate kernel-side data structures (`struct socket`, `struct file`, mbuf clusters, syncache slots, ephemeral ports) before user code observes the connection, and cannot be lifted from within the Exchange Server regardless of its I/O model. Section A.4 demonstrates that once these OS ceilings are raised, the server itself scales cleanly to the target of 70 000 connections at ~ 390 B RSS/conn and 0 % steady-state CPU (Table 2); no server-imposed ceiling was reached in the 10 k–70 k range.

A.2 Changes made to the Exchange Server

A single `listen()` argument in `exchange_server.c` was changed: `listen(server_fd, SOMAXCONN)` $\rightarrow$ `listen(server_fd, 65535)`. The C header defines `SOMAXCONN=128` on FreeBSD 14 independent of the runtime `sysctl kern.ipc.somaxconn` (raised to 65 535 in Table 1); the 128-entry syncache overflowed at 40 k and 199 of 40 000 SYNs were RST’d in a single burst (Fig. 34). The kernel clamps the argument to `kern.ipc.somaxconn`, so the new value is safe. Rows 10 k–40 k use the original build; rows from 50 k onward use the fixed build. All other server code – accept path, kqueue loop, framing, matching – is unchanged.

A.3 Client-side source-IP rotation (for $N > 55\,000$)

A single generator process from 127.0.0.1 to 127.0.0.1:5000 cannot exceed $\sim 55\,000$ connections: the TCP 4-tuple must be unique, three of its four fields are fixed, and only $\sim 55\,000$ ephemeral source ports exist. This is a client-side limit; the server sees no difference.

`idle_clients.py` was extended with three optional flags (`--src-ips`, `--src-cidr`, `--src-count`) that round-robin `s.bind((src_ip, 0))` across a pool before `connect()`, raising the single-run ceiling to $\sim K \times 55\,000$ for K source IPs. Default behaviour (used for rows 10k–50k) is unchanged.

FreeBSD does not accept `bind((127.x.x.x, 0))` unless the address is aliased on `lo0`, so the following was applied once as root:

```
ifconfig lo0 alias 127.0.0.2/32
ifconfig lo0 alias 127.0.0.3/32
ifconfig lo0 alias 127.0.0.4/32
```

Rows 60k and 70k invoke `--src-cidr 127.0.0.0/8 --src-count 4` (pool: 127.0.0.1–127.0.0.4).

A.4 Full measurement table

All rows come from `measure.sh` run against a live server holding exactly N established connections. Columns: RSS and %CPU from `ps`; server FDs from `procstat -f`; system-wide FDs from `sysctl kern.openfiles`; mbuf clusters from `netstat -m`; established count cross-checked between `sockstat` and the generator.

N	RSS (KB)	%CPU	Server FDs	Sys FDs used / max	mbuf clu. used / max	Net alloc (KB)	Established
10 000	6 140	0.0	10 009	20 094 / 300 000	2 049 / 500 000	5 644	10 000 / 10 000
20 000	9 996	0.0	20 008	40 092 / 300 000	2 051 / 500 000	5 648	19 999 / 20 000 [†]
30 000	13 968	0.0	30 009	60 176 / 300 000	2 049 / 500 000	5 644	30 000 / 30 000
40 000	17 676	0.0	39 806	79 774 / 300 000	2 050 / 500 000	5 646	39 801 / 40 000 [†]
50 000	21 888	0.0	50 009	100 197 / 300 000	2 049 / 500 000	5 644	50 000 / 50 000 [‡]
60 000	25 696	0.2	60 009	120 198 / 300 000	2 050 / 500 000	5 646	60 000 / 60 000 ^{‡§}
70 000	29 512	0.1	70 009	140 198 / 300 000	2 049 / 500 000	5 644	70 000 / 70 000^{‡§}

Table 2: Resource footprint of the Exchange Server holding N idle TCP connections. [†] Original `listen(SOMAXCONN)` build: transient RSTs during the SYN flood (1 at 20k, 199 at 40k) – see §A.2. [‡] `listen(65535)` build. [§] Source-IP rotation across 127.0.0.{1..4} (see §A.3).

Growth per 10k connections. RSS grows by ≈ 3.9 MB (~ 390 B/connection, essentially `struct ClientConnection` plus a `conns[]` slot). Server FDs grow by one per connection; system-wide FDs by two (loopback holds both ends). mbuf clusters stay at $\sim 2\,050$ across every row – idle sockets reserve socket-buffer space but do not allocate clusters until data flows. Steady-state %CPU is 0; the server thread remains parked in `kevent()` at every N (`procstat -k: sleepq_wait_sig` $\rightarrow$ `kqueue_scan` $\rightarrow$ `sys_kevent`).

A.5 Screenshots

Raw `measure.sh` outputs for three representative rows: the smallest ($N = 10\,000$), the row that exposed the listen-backlog issue ($N = 40\,000$), and the target ($N = 70\,000$). The intermediate rows are supplied as `run_20k.txt`–`run_60k.txt`.

```

root@ragnarok:/home/harsh/compiled/bonus # python3 idle_clients.py 127.0.0.1 5000 10000 --batch 1000
[+] target: 10000 idle TCP connections -> 127.0.0.1:5000
[+] client PID = 942 (use for `procstat -f 942` etc.)
[+] RLIMIT_NOFILE now soft=250000 hard=250000
[i] FreeBSD tuning hints if you hit failures at large N:
sysctl kern.maxfiles kern.maxfilesperproc kern.ipc.somaxconn
sysctl net.inet.ip.portrange.first net.inet.ip.portrange.last
sysctl kern.ipc.maxsockbuf net.inet.tcp.sendspace net.inet.tcp.recvspace
(raise them via /etc/sysctl.conf or `sudo sysctl KEY=VAL` before running)
established= 1000 failed= 0 elapsed= 0.0s rate= 29845/s
established= 2000 failed= 0 elapsed= 0.1s rate= 35584/s
established= 3000 failed= 0 elapsed= 0.1s rate= 38319/s
established= 4000 failed= 0 elapsed= 0.2s rate= 28172/s
established= 5000 failed= 0 elapsed= 0.2s rate= 23331/s
established= 6000 failed= 0 elapsed= 0.2s rate= 26964/s
established= 7000 failed= 0 elapsed= 0.2s rate= 28699/s
established= 8000 failed= 0 elapsed= 0.3s rate= 30581/s
established= 9000 failed= 0 elapsed= 0.3s rate= 32784/s
established= 10000 failed= 0 elapsed= 0.3s rate= 34183/s

[+] connect phase done: established=10000 failed=0 elapsed=0.3s
[+] client PID = 942
[+] holding 10000 sockets open. Ctrl-C to close all and exit.

While this holds, in another terminal you can run e.g.:
  sockstat -4 -c | wc -l
  netstat -an -p tcp | awk '$6=="ESTABLISHED"' | wc -l
  procstat -f <server-pid> | wc -l
  ps -o pid,rss,vsz,%cpu -p <server-pid>
  netstat -m
  sysctl kern.openfiles kern.maxfiles
[.] holding: sockets_alive=10000 pid=942 uptime=30s

```

Figure 30: $N = 10000$: ps, procstat -f, kern.openfiles. RSS=6 140 KB; server FDs=10 009; sys FDs=20 094 / 300 000.

```

root@ragnarok:/home/harsh/compiled/bonus # SPID=$(pgrep -f exchange_server | head -1)
root@ragnarok:/home/harsh/compiled/bonus # ./measure.sh $SPID 5000 | tee run_10k.txt

==== date / uname ====
Wed Sep  9 19:26:59 IST 2026
FreeBSD ragnarok 14.4-RELEASE FreeBSD 14.4-RELEASE releng/14.4-n273675-a456f852d145 GENERIC amd64

==== established connections to server port 5000 ====
ESTABLISHED_to_5000 = 0

==== server open FDs (procstat -f | wc -l) ====
server_open_fds = 10009

==== server memory / cpu (ps) ====
PID  RSS   VSZ %CPU %MEM NLWP COMMAND
941 6140 16308 0.0 0.3  1 ./exchange_server 127.0.0.1 5000

==== system-wide open files (sysctl kern.openfiles / kern.maxfiles) ====
kern.openfiles: 20094
kern.maxfiles: 300000
kern.maxfilesperproc: 250000

```

Figure 31: $N = 10000$: netstat -m + socket-buffer sysctls. 2049 mbuf clusters in use of 500 000.

```

==== mbuf / socket-buffer usage (netstat -m) ====
6186/1434/7620 mbufs in use (current/cache/total)
2849/745/2794/500000 mbuf clusters in use (current/cache/total/max)
1/507 mbuf+clusters out of packet secondary zone in use (current/cache)
0/762/762/62503 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/18519 9k jumbo clusters in use (current/cache/total/max)
0/0/0/10417 16k jumbo clusters in use (current/cache/total/max)
5644k/4896k/10541k bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 sendfile syscalls
0 sendfile syscalls completed without I/O request
0 requests for I/O initiated by sendfile
0 pages read by sendfile as part of a request
0 pages were valid at time of a sendfile request
0 pages were valid and substituted to bogus page
0 pages were requested for read ahead by applications
0 pages were read ahead by sendfile
0 times sendfile encountered an already busy page

==== socket-buffer sysctls ====
kern.ipc.maxsockbuf: 2097152
net.inet.tcp.sendspace: 32768
net.inet.tcp.recvspace: 65536
kern.ipc.somaxconn: 65535

==== server kernel stack (where is it blocked?) ====
PID   TID COMM          TDNAME          KSTACK
941 100258 exchange_server - mi_switch sleepq_catch_signals sleepq_wait_sig_sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_
kevent_generic sys_kevent amd64_syscall fast_syscall_common

==== sockstat summary for server (count) ====
server_sockets_seen_by_sockstat = 10001
root@ragnarok:/home/harsh/compiled/bonus #

```

Figure 32: $N = 10000$: server thread parked at `sys_kevent`; `sockstat=10001` (10000 clients + listener).

```

root@ragnarok:/home/harsh/compiled/bonus # python3 idle_clients.py 127.0.0.1 5000 40000 --batch 1000
[+] target: 40000 idle TCP connections -> 127.0.0.1:5000
[+] client PID = 1743 (use for 'proctstat -f 1743' etc.)
[+] RLIMIT_NOFILE now soft=250000 hard=250000
[!] FreeBSD tuning hints if you hit failures at large N:
sysctl kern.maxfiles kern.maxfilesperproc kern.ipc.somaxconn
sysctl net.inet.ip.portrange.first net.inet.ip.portrange.last
sysctl kern.ipc.maxsockbuf net.inet.tcp.sendspace net.inet.tcp.recvspace
(raise them via /etc/sysctl.conf or 'sudo sysctl KEY=VAL' before running)
established= 1000 failed= 0 elapsed= 0.0s rate= 56408/s
established= 2000 failed= 0 elapsed= 0.0s rate= 70600/s
established= 3000 failed= 0 elapsed= 0.0s rate= 69585/s
established= 4000 failed= 0 elapsed= 0.1s rate= 68877/s
established= 5000 failed= 0 elapsed= 0.1s rate= 74061/s
established= 6000 failed= 0 elapsed= 0.1s rate= 56269/s
established= 7000 failed= 0 elapsed= 0.1s rate= 57706/s
established= 8000 failed= 0 elapsed= 0.1s rate= 59527/s
established= 9000 failed= 0 elapsed= 0.2s rate= 57133/s
established= 10000 failed= 0 elapsed= 0.2s rate= 59270/s
established= 11000 failed= 0 elapsed= 0.2s rate= 59654/s
established= 12000 failed= 0 elapsed= 0.2s rate= 59005/s
established= 13000 failed= 0 elapsed= 0.2s rate= 59829/s
established= 14000 failed= 0 elapsed= 0.2s rate= 60456/s
established= 15000 failed= 0 elapsed= 0.3s rate= 54245/s
established= 16000 failed= 0 elapsed= 0.3s rate= 48347/s
established= 17000 failed= 0 elapsed= 0.3s rate= 48999/s
established= 18000 failed= 0 elapsed= 0.4s rate= 49408/s
established= 19000 failed= 0 elapsed= 0.4s rate= 49735/s
established= 20000 failed= 0 elapsed= 0.4s rate= 49798/s
established= 21000 failed= 0 elapsed= 0.4s rate= 50688/s
established= 22000 failed= 0 elapsed= 0.4s rate= 51873/s
established= 23000 failed= 0 elapsed= 0.4s rate= 52265/s
established= 24000 failed= 0 elapsed= 0.5s rate= 52914/s
established= 25000 failed= 0 elapsed= 0.5s rate= 52703/s
established= 26000 failed= 0 elapsed= 0.5s rate= 52336/s
established= 27000 failed= 0 elapsed= 0.5s rate= 53299/s
established= 28000 failed= 0 elapsed= 0.5s rate= 53463/s

```

Figure 33: $N = 40000$ (original SOMAXCONN build): RSS=17 676 KB; server FDs=39 806; sys FDs=79 774 / 300 000.

```

established= 18000 failed= 0 elapsed= 0.4s rate= 49408/s
established= 19000 failed= 0 elapsed= 0.4s rate= 49735/s
established= 20000 failed= 0 elapsed= 0.4s rate= 49798/s
established= 21000 failed= 0 elapsed= 0.4s rate= 50688/s
established= 22000 failed= 0 elapsed= 0.4s rate= 51873/s
established= 23000 failed= 0 elapsed= 0.4s rate= 52265/s
established= 24000 failed= 0 elapsed= 0.5s rate= 52914/s
established= 25000 failed= 0 elapsed= 0.5s rate= 52703/s
established= 26000 failed= 0 elapsed= 0.5s rate= 52336/s
established= 27000 failed= 0 elapsed= 0.5s rate= 53299/s
established= 28000 failed= 0 elapsed= 0.5s rate= 53463/s
established= 29000 failed= 0 elapsed= 0.5s rate= 54116/s
established= 30000 failed= 0 elapsed= 0.5s rate= 54611/s
[!] connect #30994 failed: 54 Connection reset by peer
[!] connect #30995 failed: 54 Connection reset by peer
[!] connect #30996 failed: 54 Connection reset by peer
[!] connect #30997 failed: 54 Connection reset by peer
[!] connect #30998 failed: 54 Connection reset by peer
established= 30994 failed= 6 elapsed= 0.6s rate= 55267/s
established= 31801 failed= 199 elapsed= 0.6s rate= 55246/s
established= 32801 failed= 199 elapsed= 0.6s rate= 54801/s
established= 33801 failed= 199 elapsed= 0.6s rate= 54668/s
established= 34801 failed= 199 elapsed= 0.7s rate= 53424/s
established= 35801 failed= 199 elapsed= 0.7s rate= 52361/s
established= 36801 failed= 199 elapsed= 0.7s rate= 51445/s
established= 37801 failed= 199 elapsed= 0.7s rate= 50803/s
established= 38801 failed= 199 elapsed= 0.8s rate= 50699/s
established= 39801 failed= 199 elapsed= 0.8s rate= 50580/s

[+] connect phase done: established=39801 failed=199 elapsed=0.8s
[+] first failure was at attempt #30994
[+] client PID = 1743
[+] holding 39801 sockets open. Ctrl-C to close all and exit.

```

Figure 34: $N = 40\,000$: generator log – 199 ECONNRESET failures in one burst from attempt #30 994 (syncache overflow against SOMAXCONN=128; fixed per §A.2).

```

root@ragnarok:/home/harsh/compiled/bonus # ./measure.sh $SPID 5000 | tee run_40k.txt

==== date / uname ====
Wed Sep  9 19:54:58 IST 2026
FreeBSD ragnarok 14.4-RELEASE FreeBSD 14.4-RELEASE releng/14.4-n273675-a456f852d145 GENERIC amd64

==== established connections to server port 5000 ====
ESTABLISHED_to_5000 = 0

==== server open FDs (procstat -f | wc -l) ====
server_open_fds = 39806

==== server memory / cpu (ps) ====
  PID  RSS  VSZ %CPU %MEM NLWP COMMAND
 1734 17676 32180  0.0  0.9   1 ./exchange_server 127.0.0.1 5000

==== system-wide open files (sysctl kern.openfiles / kern.maxfiles) ====
kern.openfiles: 79774
kern.maxfiles: 300000
kern.maxfilesperproc: 250000

```

Figure 35: $N = 40\,000$: netstat -m – 2 050 mbuf clusters in use, essentially unchanged from 10 k.


```

==== mbuf / socket-buffer usage (netstat -m) ====
6186/1944/8130 mbufs in use (current/cache/total)
2050/1252/3302/500000 mbuf clusters in use (current/cache/total/max)
2/760 mbuf+clusters out of packet secondary zone in use (current/cache)
0/762/762/62503 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/18519 9k jumbo clusters in use (current/cache/total/max)
0/0/0/18417 16k jumbo clusters in use (current/cache/total/max)
5646k/6038k/11684k bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 sendfile syscalls
0 sendfile syscalls completed without I/O request
0 requests for I/O initiated by sendfile
0 pages read by sendfile as part of a request
0 pages were valid at time of a sendfile request
0 pages were valid and substituted to bogus page
0 pages were requested for read ahead by applications
0 pages were read ahead by sendfile
0 times sendfile encountered an already busy page

==== socket-buffer sysctls ====
kern.ipc.maxsockbuf: 2097152
net.inet.tcp.sendspace: 32768
net.inet.tcp.recvspace: 65536
kern.ipc.somaxconn: 65535

==== server kernel stack (where is it blocked?) ====
PID    TID COMM                TDNAME                KSTACK
1734 100316 exchange_server - mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_
kevent_generic sys_kevent amd64_syscall fast_syscall_common

==== sockstat summary for server (count) ====
server_sockets_seen_by_sockstat = 39798
root@ragnarok:/home/harsh/compiled/bonus # clear

```

Figure 36: $N = 40\,000$: kernel stack still at sys_kevent at 39 801 established connections; %CPU=0.0.

```

root@ragnarok:/home/harsh/compiled/bonus # python3 idle_clients.py 127.0.0.1 5000 70000 --batch 2000 --src-cidr 127.0.0.0/8 --src-count 4
[+] target: 70000 idle TCP connections -> 127.0.0.1:5000
[+] client PID = 4127 (use for 'proctstat -f 4127' etc.)
[+] rotating source IPs over pool of 4: 127.0.0.1, 127.0.0.2, 127.0.0.3, 127.0.0.4
[+] theoretical single-run ceiling raised to ~220,000 connections
[+] RLIMIT_NOFILE now soft=250000 hard=250000
[i] FreeBSD tuning hints if you hit failures at large N:
sysctl kern.maxfiles kern.maxfilesperproc kern.ipc.somaxconn
sysctl net.inet.ip.portrange.first net.inet.ip.portrange.last
sysctl kern.ipc.maxsockbuf net.inet.tcp.sendspace net.inet.tcp.recvspace
(raise them via /etc/sysctl.conf or 'sudo sysctl KEY=VAL' before running)
established= 2000 failed= 0 elapsed= 0.1s rate= 19864/s
established= 4000 failed= 0 elapsed= 0.1s rate= 27084/s
established= 6000 failed= 0 elapsed= 0.2s rate= 33101/s
established= 8000 failed= 0 elapsed= 0.2s rate= 37952/s
established= 10000 failed= 0 elapsed= 0.2s rate= 40552/s
established= 12000 failed= 0 elapsed= 0.3s rate= 43560/s
established= 14000 failed= 0 elapsed= 0.3s rate= 47621/s
established= 16000 failed= 0 elapsed= 0.3s rate= 49160/s
established= 18000 failed= 0 elapsed= 0.4s rate= 50737/s
established= 20000 failed= 0 elapsed= 0.4s rate= 53072/s
established= 22000 failed= 0 elapsed= 0.4s rate= 52285/s
established= 24000 failed= 0 elapsed= 0.5s rate= 51673/s
established= 26000 failed= 0 elapsed= 0.5s rate= 53103/s
established= 28000 failed= 0 elapsed= 0.5s rate= 53696/s
established= 30000 failed= 0 elapsed= 0.6s rate= 54280/s
established= 32000 failed= 0 elapsed= 0.6s rate= 55546/s
established= 34000 failed= 0 elapsed= 0.6s rate= 56288/s
established= 36000 failed= 0 elapsed= 0.7s rate= 54817/s
established= 38000 failed= 0 elapsed= 0.8s rate= 50160/s
established= 40000 failed= 0 elapsed= 0.8s rate= 49586/s
established= 42000 failed= 0 elapsed= 0.9s rate= 48847/s
established= 44000 failed= 0 elapsed= 0.9s rate= 46729/s
established= 46000 failed= 0 elapsed= 1.0s rate= 44407/s
established= 48000 failed= 0 elapsed= 1.1s rate= 43836/s
established= 50000 failed= 0 elapsed= 1.2s rate= 43246/s

```

Figure 37: $N = 70\,000$: RSS=29 512 KB; server FDs=70 009; sys FDs=140 198 / 300 000 (47% of cap).

```

established= 50000 failed= 0 elapsed= 1.2s rate= 43210/s
established= 52000 failed= 0 elapsed= 1.2s rate= 42460/s
established= 54000 failed= 0 elapsed= 1.3s rate= 41736/s
established= 56000 failed= 0 elapsed= 1.4s rate= 41293/s
established= 58000 failed= 0 elapsed= 1.4s rate= 41115/s
established= 60000 failed= 0 elapsed= 1.5s rate= 40416/s
established= 62000 failed= 0 elapsed= 1.5s rate= 40177/s
established= 64000 failed= 0 elapsed= 1.6s rate= 40033/s
established= 66000 failed= 0 elapsed= 1.7s rate= 39810/s
established= 68000 failed= 0 elapsed= 1.7s rate= 39168/s
established= 70000 failed= 0 elapsed= 1.8s rate= 38592/s

[+] connect phase done: established=70000 failed=0 elapsed=1.8s
[+] client PID = 4127
[+] holding 70000 sockets open. Ctrl-C to close all and exit.

While this holds, in another terminal you can run e.g.:
    sockstat -4 -c | wc -l
    netstat -an -p tcp | awk '$6=="ESTABLISHED"' | wc -l
    procstat -f <server-pid> | wc -l
    ps -o pid,rss,vsz,%cpu -p <server-pid>
    netstat -m
    sysctl kern.openfiles kern.maxfiles
[.] holding: sockets_alive=70000 pid=4127 uptime=32s
[.] holding: sockets_alive=70000 pid=4127 uptime=62s

```

Figure 38: $N = 70\,000$: 70 000 established, 0 failed under source-IP rotation and the listen(65535) build.

```

Problems Output Debug Console Terminal Ports
root@ragnarok:/home/harsh/compiled/bonus # ./measure.sh $SPID 5000 | tee run_70k.txt

==== date / uname ====
Wed Sep  9 20:40:32 IST 2026
FreeBSD ragnarok 14.4-RELEASE FreeBSD 14.4-RELEASE releng/14.4-n273675-a456f852d145 GENERIC amd64

==== established connections to server port 5000 ====
ESTABLISHED_to_5000 = 0

==== server open FDs (procstat -f | wc -l) ====
server_open_fds = 70009

==== server memory / cpu (ps) ====
  PID  RSS   VSZ %CPU %MEM NLWP  COMMAND
4117 29512 45492  0.1  1.4   1  ./exchange_server 127.0.0.1 5000

==== system-wide open files (sysctl kern.openfiles / kern.maxfiles) ====
kern.openfiles: 140198
kern.maxfiles: 300000
kern.maxfilesperproc: 250000

```

Figure 39: $N = 70\,000$: 2 049 mbuf clusters in use, identical to the 10 k row.

```

==== mbuf / socket-buffer usage (netstat -m) ====
6186/1719/7985 mbufs in use (current/cache/total)
2849/1587/3556/580000 mbuf clusters in use (current/cache/total/max)
1/587 mbuf+clusters out of packet secondary zone in use (current/cache)
0/762/762/62583 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/18519 9k jumbo clusters in use (current/cache/total/max)
0/0/0/18417 16k jumbo clusters in use (current/cache/total/max)
5644K/6491K/12136K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 sendfile syscalls
0 sendfile syscalls completed without I/O request
0 requests for I/O initiated by sendfile
0 pages read by sendfile as part of a request
0 pages were valid at time of a sendfile request
0 pages were valid and substituted to bogus page
0 pages were requested for read ahead by applications
0 pages were read ahead by sendfile
0 times sendfile encountered an already busy page

==== socket-buffer sysctls ====
kern.ipc.maxsockbuf: 2097152
net.inet.tcp.sendspace: 32768
net.inet.tcp.recvspace: 65536
kern.ipc.somaxconn: 65535

==== server kernel stack (where is it blocked?) ====
PID   TID  COMM             TDNAME          KSTACK
4117  180252 exchange_server -             mi_switch sleepq_catch_signals sleepq_wait_sig_sleep kqueue_scan kqueue_kevent kern_kevent_fp kern_
kevent_generic sys_kevent amd64_syscall fast_syscall_common

==== sockstat summary for server (count) ====
server_sockets_seen_by_sockstat = 78001
root@ragnarok: /home/harsh/compiled/bonus #

```

Figure 40: $N = 70\,000$: sockstat=70001; server thread at sys_kevent; %CPU=0.1 (tail of connect flood).

A.6 Answers to Section 6.9 questions

Q1. At each connection count, is the server able to maintain all requested connections? If not, at which count does it first fail?

Answer. Yes at every measured count. The server maintained 10 k/20 k/30 k/40 k/50 k/60 k/70 k simultaneous idle TCP connections (Table 2). Two transient RST bursts occurred during the connect flood (1 at 20 k, 199 at 40 k), both caused by the C-header SOMAXCONN=128 capping the syncache below the tuned `kern.ipc.somaxconn`=65 535. After the fix (§A.2) runs at 50 k / 60 k / 70 k completed with zero failures. No first-fail count was reached; at $N = 70\,000$ the server was idle at 0 % CPU with ~47 % of the system-wide FD cap unused.

Q2. What is the first significant bottleneck?

Answer. Kernel socket-structure and socket-buffer-cluster pools, not application resources. With FreeBSD defaults the first two ceilings hit were:

1. `kern.ipc.maxsockets` ($\approx 64\,177$): loopback consumes two kernel sockets per connection, so the effective ceiling is $\sim 32\,000$ – exactly where `ECONNRESET` first appeared in an untuned run.
2. `kern.ipc.nmbclusters` ($\approx 125\,007$): next failure mode is `ENOBUFS` on `socket()`.

Both are boot-time tunables (Table 1); runtime caps on `kern.maxfiles`, `kern.maxfilesperproc`, `kern.ipc.somaxconn` and `net.inet.ip.portrange.*` were also raised. Once these were lifted the server itself never bounded scaling in the 10 k–70 k range (RSS <30 MB, %CPU=0, mbuf clusters flat at $\sim 2\,050$). Next-ceiling ordering, without further tuning: `maxsockets` $\rightarrow$ `nmbclusters` $\rightarrow$ FD limits $\rightarrow$ generator-side ephemeral ports ($\sim 55\,k$ per source IP; addressed in §A.3) $\rightarrow$ reserved socket-buffer memory ($\sim 96\,KB \times 2$ sockets/conn).

Q3. How does the concurrency / I/O design contribute to the observed bottleneck?

Answer. Negligibly. The server uses a single-threaded event loop over `kqueue` with non-blocking sockets:

- One thread, always parked in `kevent()` (`procstat -k` at every N): no per-connection stack, no `fd_set` scan, no busy-poll; %CPU=0 when idle.
- Per-connection application state is ~ 360 B (`struct ClientConnection`) plus one pointer in `conns[]`; write buffer is NULL until data is queued. Measured RSS scaled at ~ 390 B/conn.
- `conns[]` grows by doubling from `MIN_CAPACITY=10`; at 70 k it occupies only ~ 640 KB.

The design has no $O(N)$ work per event and no per-connection resource beyond that small struct; the Q2 bottlenecks are kernel-side allocations that any correct TCP server incurs.

Q4. Would changing the I/O / concurrency mechanism help with the bottleneck identified?

Answer. No. The Q2 bottlenecks are kernel-side per-connection allocations paid by any correct TCP server regardless of user-space multiplexing:

- Thread-per-connection: each pthread costs $\sim 1\text{--}2$ MB stack, a `struct thread`, and a slot in `kern.threads.max_threads_per_proc` (~ 1500); collapses at $O(\text{few thousand})$ connections.
- `select()`: capped at `FD_SETSIZE=1024` with $O(N)$ scans per event.
- `poll()`: no `FD_SETSIZE` cap but still $O(N)$ scans per event; higher CPU than `kqueue` at 70 k.

Alternatives either hit the same kernel-resource ceiling with more overhead or fail earlier for their own reasons.

Q5. Quantify the trade-off – if you change your implementation or configuration, measure the relevant resource again and compare.

Answer. The `listen()` backlog (§A.2) was changed with all other tuning fixed. Metric: successful vs. failed connects during the flood.

Regime	Setting	Established / requested	Lost
Before	<code>listen(fd, SOMAXCONN)</code> , <code>SOMAXCONN=128</code>	39 801 / 40 000	199 (0.50 %)
After	<code>listen(fd, 65535)</code>	50 000 / 50 000	0 (0.00 %)

At higher load, the one-line change eliminated the losses; 60 k and 70 k subsequently completed with 0 failures (Table 2). Runtime cost of the change is zero: the backlog only caps syn-cache occupancy, and no eager allocation occurs.

A.7 Notes and caveats

- **Loopback doubles the kernel-socket cost.** Each connection holds two kernel sockets because both endpoints run on the same VM; a two-host deployment halves the `kern.ipc.maxsockets` pressure at the same N .
- **The 40 k row is retained** with its 199 losses to serve as direct evidence for the `SOMAXCONN` issue and as the A/B pair for Q5.
- **Idle vs. active.** All rows measure idle connections per the handout; active traffic would allocate mbuf clusters in proportion to in-flight bytes.

- **Non-zero %CPU at 60 k / 70 k** (0.2 / 0.1) is the tail of the connect flood, not steady state; `procstat -k` confirms the server was back in `sys_kevent`.